高级语言程序设计

第4章：对象和类

计算机学院 陈建全
Email: cjq@nwpu.edu.cn

2024/2/8

- ◆4.1 对象、实例方法和实例变量
- ◆4.2 构造函数和对象初始化
- ◆4.3 Java语言内置类
- ◆4.4 类程序设计例子
- ◆4.5 类的继承
- ◆4.6 枚举类型

4.1.1 对象、实例方法和实例变量

- 类是对象的抽象，或者更准确地说，类的非静态部分用来描述对象。从编程的角度来看，类是用来创建对象的。类是一种用于构造程序的基本单元。
- 类的非静态部分指明或描述了对象将包含哪些变量和方法。
- 类是对象的蓝图：类定义了一个对象的结构和行为，包括它包含的变量（字段、属性）和方法（行为）。你可以将类比作建筑图纸，而对象则是根据图纸建造的房子。每个对象都是由其对应类的构造函数创建的具体实例。
- 类是对象的类型：在类型系统中，类定义了一种类型，而对象是这种类型的一个实例。这意味着，类提供了用来生成相同类型对象的特定模板。

4.1.1 对象、实例方法和实例变量

- 对象是类的实例化：当你使用new关键字和类的构造器创建一个新对象时，实际上是在实例化类。这个过程会根据类的定义分配内存空间，并初始化对象的状态。
- 类是静态的，对象是动态的：类是编写代码时定义的，它不会改变，并在整个应用程序的生命周期中一直存在。与之相对的是，对象在程序运行时被创建、使用和销毁，所以它们是动态的。存在于内存中的对象可以在运行时发生改变，而类的定义则不会发生改变。

4.1.1 对象、实例方法和实例变量

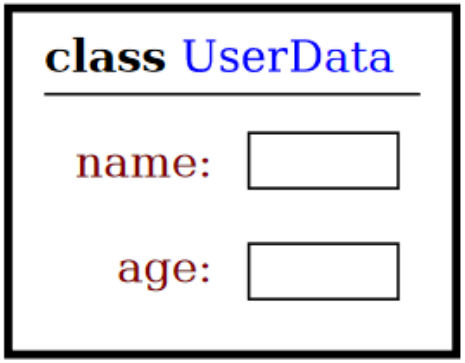
◆类定义时：

■使用 static修饰符的变量和方法叫类变量和类方法。

■没有 static 描述的变量和方法称为实例变量和实例方法。

◆类变量和类方法是属于类的一部分。比如下面类定义：

```
class UserData {
    static String name;
    static int age;
}
```



■name和age是UserData类的一部分,它们的全名是UserData.name和UserData.age。

■程序运行时，他们在内存中只有一个拷贝。像右图那样。

■如果用来表示用户，只能表示一个用户。

4.1.1 对象、实例方法和实例变量

◆再看这个类：

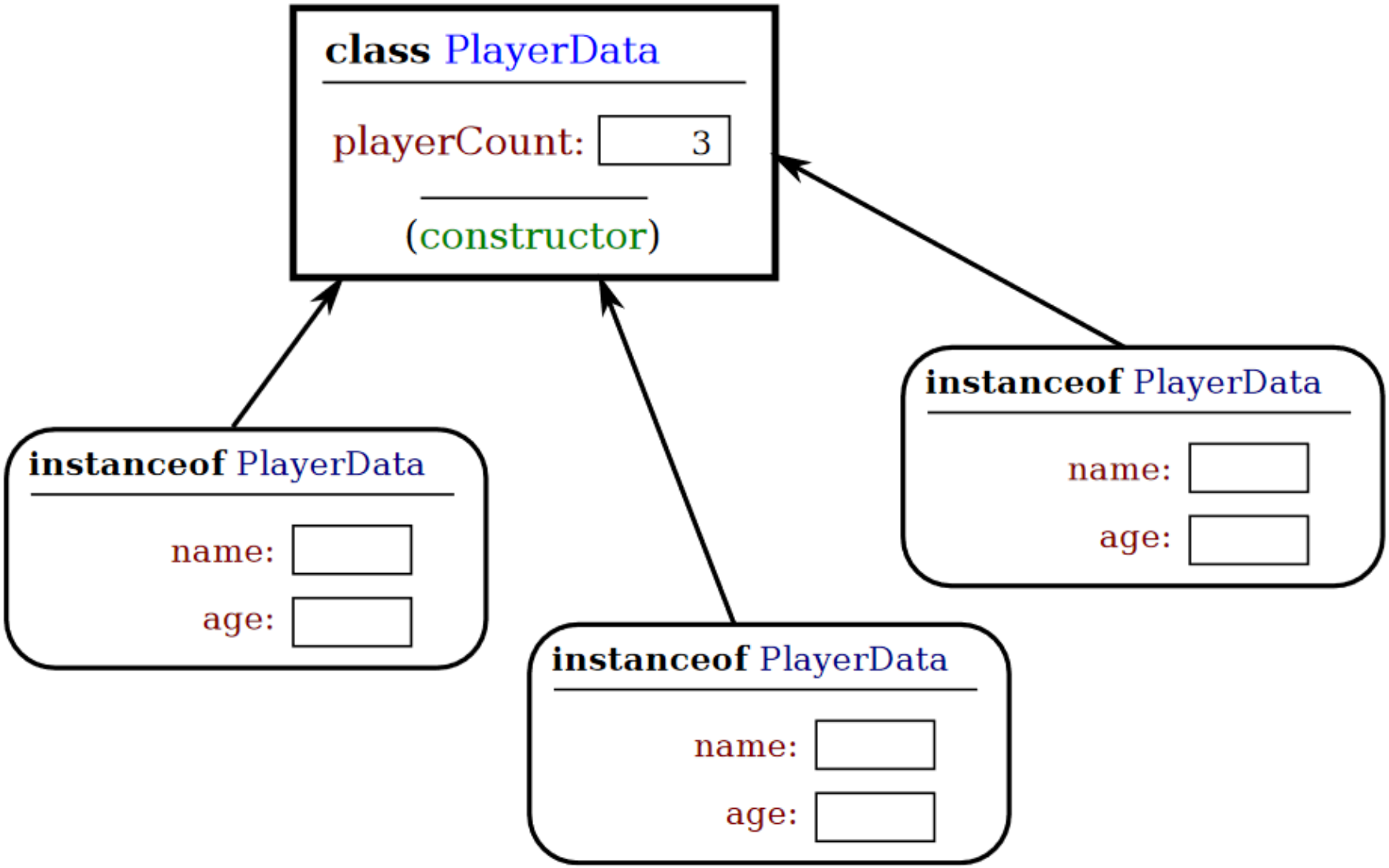
■PlayerData.playerCount属于类，类加载后在内 存中只有一个拷贝。

■但是name和age是实例变量，它们只能属于某个类的实例。不存在PlayerData.name和PlayerData.age. 只有利用类创建对象(实例化)后，它们才会存在于对象的内存中。

```
class PlayerData {
    static int playerCount;
    String name;
    int age;
}
```

4.1.1 对象、实例方法和实例变量

◆利用PlayData创建三个对象后的内存情况。



4.1.1 对象、实例方法和实例变量

◆说明:

- 内存中只有一个PlayerCount
- 每个对象都有自己的name和age. 利用类创建的对象叫做类的实例。
- 图中PlayerData类内部还有一个“构造方法（constructor）“。构造方法是用来创建对象的。

◆程序运行时:

- 每加入一个玩家，就为这个玩家创建一个对象，每个对象有自己的name和age。
- 每退出一个玩家，就把这个玩家对应的对象销毁。
- 这样就可以动态模拟游戏中发生的情况，但是静态变量无法做到以上的添加和销毁过程。

- 方法的工作方式类似于变量。
  - 静态方法是类的一部分，非静态或实例方法是对象的一部分。
  - 虽然程序运行时不是为每个对象都保留一个实例方法的编译代码的副本，但是从逻辑上讲，实例方法是对象的一部分。

◆（1）引用变量和对象实例

■类是一种类型，在声明变量时可以指定类作为类型。比如对于以下类：

```
public class Student {
   public String name; // 学生姓名.
   public double test1, test2, test3; // 3 门课的考试成绩.
   public double getAverage() { // 计算平均成绩
       return (test1 + test2 + test3) / 3;
   }
} // end of class Student
```

就可以这样声明一个变量std:

```
Student std;
```

4.1.2 对象基础

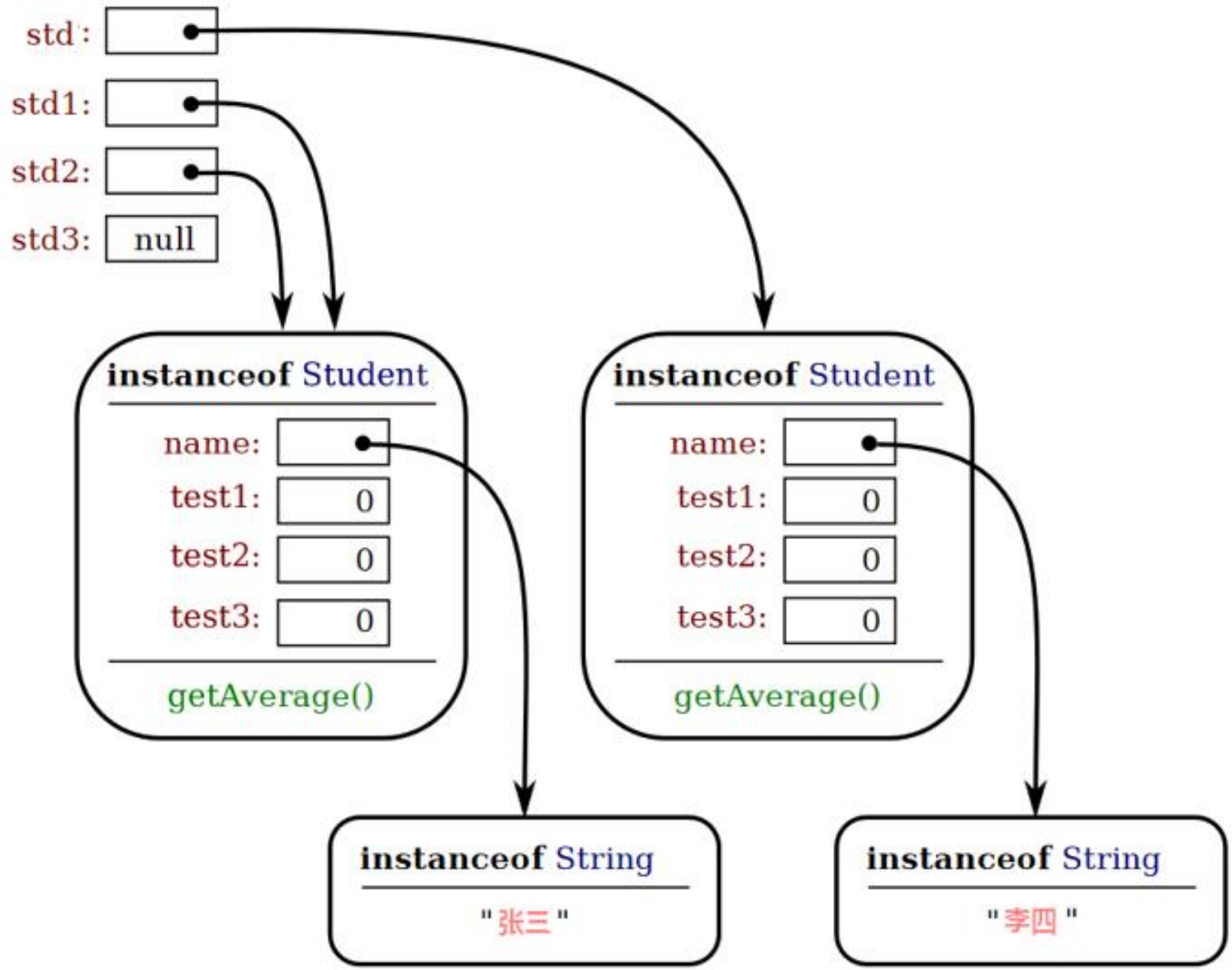
- 声明一个变量并不会创建一个对象，对象需要通过new进行创建。
- 对象创建后在堆空间中，变量的引用值指向相应的对象。
- 变量也不可能用来保存对象，变量只能用来保存对象的引用。
  - `std = new Student();`
  - 正确的描述：变量 std 引用到对象，或变量std指向对象
- std引用所引用对象的实例变量可以描述为 std.name、std.test1、std.test2 和 std.test3。
- 类似地，也可以通过std.getAverage()来调用对象中的 getAverage()实例方法, 取得这个学生的平均成绩。

- std是一个引用变量，当不引用对象时，可以把它的值置为null.
  - `std = null;`
- 当std的值为null时，std.test1会引起异常NullPointerException
- 还有，name也是一个引用类型，指向的的类型是String。

■比如下面代码执行后，内存情况如下页所示：

```
Student std, std1, std2, std3; // 定义了是个类型为 Student 类的变量
std = new Student(); // 创建 Student 类的对象，并将对象的引用保存在 std 变量中
std1 = new Student(); //创建 Student 类的对象，并将对象的引用保存在 std1 变量中
std2 = std1; //将 std1 中保存的引用复制到 std2 中
std3 = null; //将 std1 变量值设置为 null
std.name = "张三"; //设置 std 指向的对象的成员 name 值为“张三”
std1.name = "李四";
//其它成员变量的值设置为缺省值
```

4.1.2 对象基础


std:
std1:
std2:
std3:
null
instanceof Student
name:
test1:
0
test2:
0
test3:
0
getAverage()
instanceof Student
name:
test1:
0
test2:
0
test3:
0
getAverage()
instanceof String
"张三"
instanceof String
"李四"

- （2） 引用变量赋值
  - 引用变量的赋值，只是复制变量引用的地址，而不会复制对象本身。因此：
  - std2=std1语句执行的结果，就是图中所示的，两个变量指向同一个对象。
- （3）对象的同一和等值判断
  - 可以使用运算符==和!=测试对象是否相等，即检查是否为同一个对象。
  - 判断条件 “if (std1== std2)”执行时，程序检查存储在变量 std1 和 std2 中的值是否相同， 也就是对象的地址是否相等。这就是”同一判断 “
  - 如果两个引用变量地址相等，就说两个变量是”同一的(identical) “。

◆（3）对象的同一和等值判断

■有时需要判断两个对象的每个成员变量值是否都一样，也就是：

```
std1.test1 == std2.test1 && std1.test2 ==
std2.test2 && std1.test3 == std2.test3 &&
std1.name.equals(std2.name)
```

这就是”等值判断“。

■字符串的等值判断，需要用String类的equals()方法进行。比如判断std1的姓名是否叫”张三“，判断如下：

```
if (std1.name.equals("张三"))
```
(√)

不能这样判断：

```
if (std1.name=="张三")
```
(×)

■ Java的很多内置类都提供了自己的equals()方法进行等值判断。比如String，Integer。

◆再看一个例子:

```
public class E0401StringPool {
    public static void main(String args[])
    {
        String a = "Hello";
        String b = "Hello";
        String c = new String("Hello");
        System.out.println("a==b is "+(a==b));
        System.out.println("a==c is "+(a==c));
        System.out.println("a.equals(c) is
"+a.equals(c));
    }
}
```

输出:

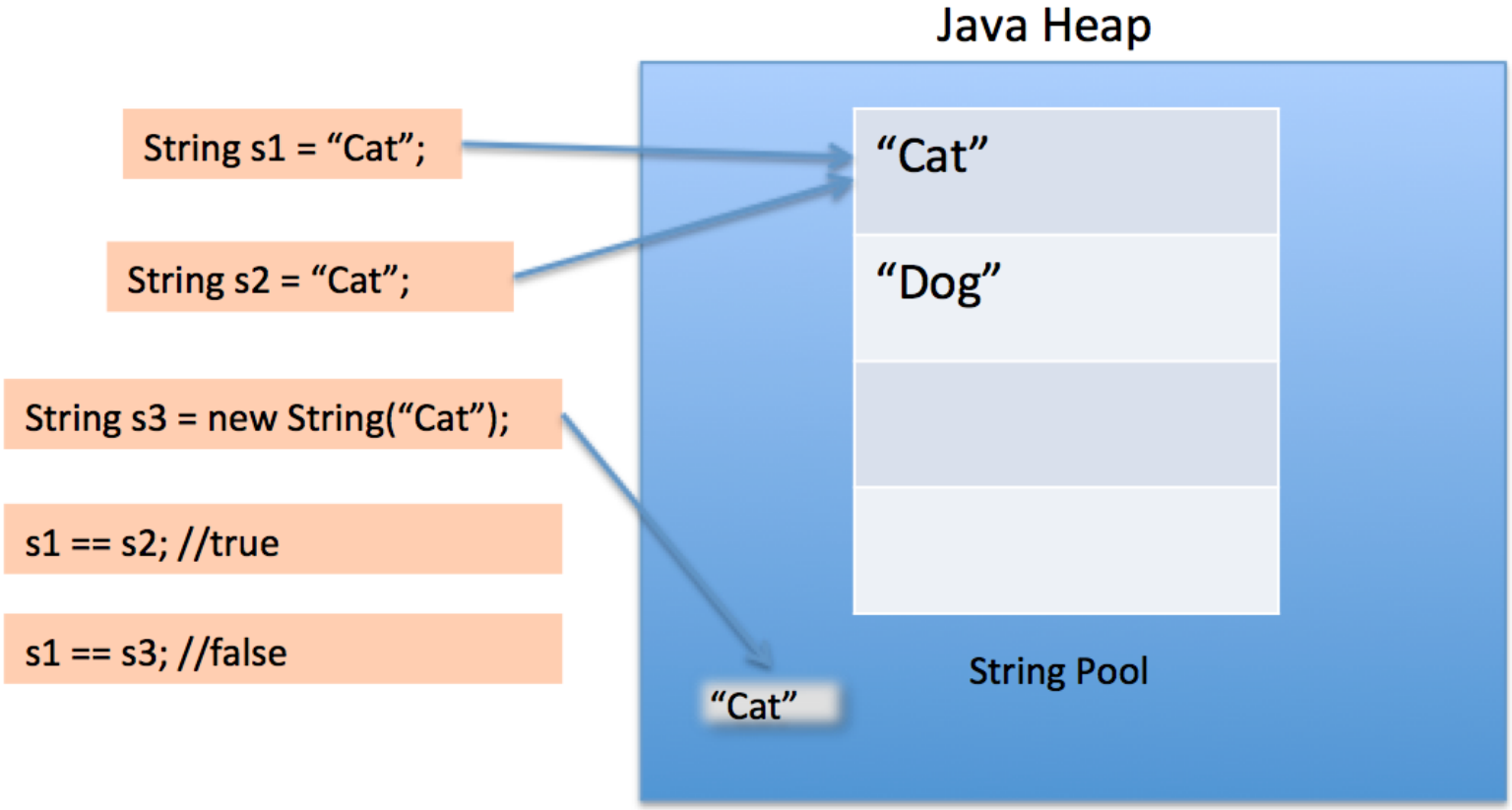
a==b is true

a==c is false

a.equals(c) is true

◆例子说明

双引号中的字符串，作为特殊的对象保存在字符串池中。而且重复的字符串只会保存一份。这叫“字符串池化”(**String Interning**)； **s3那一行new创建字符串时，会在堆中分配内存。**



◆例子的进一步解读*

更进一步了解字符串，需要了解字符串的intern方法的原理。可以参考：

https://stackoverflow.com/questions/10578984/what-is-java-string-interning

https://www.geeksforgeeks.org/interning-of-string/

◆ (4) 终结变量

■变量声明为 final时，一旦变量被初始化，存储在变量中的值就永远不能改变。(对比C语言的const变量)

■变量是最终的，而对象不是最终的。以下代码完全合法：

```
final Student stu = new Student();
stu.name = "张三";
```

□stu不允许重新赋值，但是stu.name重新赋值没有问题。

■引用变量作为参数时，传递的是实参的地址。

□这样函数内部如果修改了实参引用的对象，则实参对应的独显会被修改。

□函数内部对参数的引用变量重新赋值，不影响实参。

□如果参数是final修饰的，函数不允许对参数引用值赋值。

```
public class E0402ReferenceParameter {
    private static void doChange(Student s){
        s.name="李四";
    }
    private static void dontChange(Student s){
        s = new Student();
        s.name="李五";
    }
    private static void dontChange2(final Student
s){
        //下面语句编译错误: The final local variable
s cannot be assigned. It must be blank and not
using a compound assignment
        //s = new Student();
        s.name="李六";
    }
```

```
    public static void main(String[] args) {
        Student stu = new Student();
        stu.name = "张三";
        doChange(stu);
        System.out.println(stu.name); //李四
        dontChange(stu);
        System.out.println(stu.name); //李四
        dontChange2(stu);
        System.out.println(stu.name); //李六
    }

}
 class Student {
    public String name; // 学生姓名.
    public double test1, test2, test3; // 3 门课的考试成绩.
    public double getAverage() { // 计算平均成绩
        return (test1 + test2 + test3) / 3;
     }
} // end of class Student
```

```
$java -Dfile.encoding=UTF-8 E0309ReferenceParameter.java
李四
李四
李六
```

- doChange内部，通过形参s,修改了s.name。实参对应的name也被改变。（对比c语言指针参数）
- dontChange对形参s重新赋值，不影响实参。
- dontChange2的参数由final修饰，因此在函数内部不允许对stu赋值。

- final关键字修饰变量时：
  - 类成员变量：当你将一个类的成员变量声明为 final，这意味着一旦它被赋予了初始值之后（这通常发生在构造器中或者是在声明时直接初始化），它的值就不可更改。
  - 局部变量：在方法内部声明的变量可以被声明为 final。一旦局部变量被赋予一个初始值后，它就不能被再次赋值。
  - 方法参数：方法的参数可以被声明为 final。当一个参数被声明为 final，意味着你不能在方法体内部更改这个参数的引用值。

4.1.3 Java 的访问控制

- 访问控制(Access Control)通过限制对类的内部结构和工作原理的访问，直接支持了封装的概念，是实现封装的一种机制。
- 通过使用不同的访问修饰符，我们可以控制类的内部数据和实现细节的暴露程度，从而保护对象的状态不被不恰当的外部访问和修改，这正是封装的核心理念。
- Java 提供 `public`、`protected`、`default`（代码前不添加任何修饰符）和 `private`四种访问级别。从安全的角度看，“`private`”最高，只能被类本身访问；`public` 最低，可以被外界任意使用。

4.1.3 Java 的访问控制

◆顶级类（即不是内部类或嵌套类的类）来说，只能使用 public 或默认（也叫包访问权限，就是没有指定修饰符）两种。

类型	无修饰	public
同一包中的类	yes	yes
不同包中的类	no	yes

◆对于内部类、嵌套类和成员（如方法和变量）来说，则可以使用所有四种访问控制级别

4.1.3 Java 的访问控制

- 类成员的访问控制
  - 类的可见性为外层边界, 成员的可见性首先受制于类的可见性。
- （1）包访问:
  - 如果不指定访问修饰符，那么成员、类默认采用包访问级别。也就是仅允许同一个包内的访问；
  - 被称为“包访问权限(package-private)”或者“默认访问权限（ default access)”。
  - 由于一个编译单元（一个文件）只能属于一个包，因此一个编译单元中的所有类都可以通过包访问权限获得相互访问。

◆比如，同一个目录下有两个Java文件：

```
//InputBox.java
class InputBox {
    void f() {
        System.out.println("InputBox.f()"); }
}
```

```
//Application.java
class Application {
    public static void main(String[] args) {
        InputBox x = new InputBox();
        x.f();
    }
}
```

◆(2) 公有(public)

■可以被其他任何类访问(前提是对类成员所在的类有访问权限)

■比如， access/structure/Linestructure.java如下：

```
package access.structure;
public class LineStructure {
    public LineStructure() {
        System.out.println("LineStructure constructor");
    }
    void show() { System.out.println("show"); }
}
```

◆(2) 公有(public)

■使用LineStructure类的代码如下:

```
import access.structure.*;
public class App {
    public static void main(String[] args) {
        LineStructure x = new LineStructure();
        //x.show(); // 不能访问
    }
}
```

```
C>tree /f
文件夹 PATH 列表
卷序列号为 6A8A-A920
C:.
    App.java

└─access
    └─structure
            LineStructure.java
```

◆如果是命令行编译

- 应该把当前目录设置到App.java所在的目录。
- 检查classpath环境变量。如果没有设置，默认就会搜索当前目录。
- 否则如果设置了classpath,必须包含".",不然编译时会找不到符号" LineStructure "。

```
C>set classpath
环境变量 classpath 没有定义

C>javac App.java

C>java App
LineStructure constructor
```

- (3)私有(private)
  - 只能被这个类本身访问，在类外不可见。即使是同一个包中的类也不允许访问，除非通过公共的类方法（例如 getter/setter 方法）。private 是最严格的访问级别，确保了类的封装性。
  - 比如如下代码：
    - 类RedBlackTree的构造方法是private修饰的，因此无法从类外直接使用new创建该类的对象。
    - 同时提供一个makeARedBlackTree方法，允许类外利用这个方法创建类的对象。

◆(3)私有(private)

```
class RedBlackTree {
    private RedBlackTree() {}
    static RedBlackTree makeARedBlackTree() {
        return new RedBlackTree();
    }
}
public class E0405PrivateAccess {
    public static void main(String[] args) {
        //! RedBlackTree x = new RedBlackTree();
        RedBlackTree x =
RedBlackTree.makeARedBlackTree();
    }
}
```

- (3)私有(private)
  - 只能在类内部使用的方法用private修饰，可称为”Helper method “。
    - 相应的一个Helper类通常是指包含一组静态方法的类，这些方法为其他类提供辅助功能，常用于执行通用任务，如数学计算、字符串处理、文件操作等。*
  - 类中的 private 成员变量，其访问特性也是如此。Java 程序设计中，除非必须公开的成员，否则应该将所有成员设为私有的(细节隐藏）。

◆(3)私有(private)

把构造函数设置为private, 有几种常见的用途和设计模式:

- ■静态工厂方法：私有构造函数还可以与静态工厂方法配合使用，静态工厂方法可以在需要时创建对象，并且可以更灵活地控制对象的创建过程。
- ■工具类：私有构造函数可以用于包含静态方法的工具类。这样的类通常不需要实例化，因此将构造函数设为私有可以避免意外实例化该类。
- ■单例模式：私有构造函数可以用于实现单例模式，确保只有一个实例可以被创建。在单例模式中，类只有一个实例，并提供一个访问该实例的静态方法。通过私有构造函数，防止外部代码直接创建实例，从而保证单例的唯一性。

- (4)保护(protected)
  - 可以被同一个包中的其他类访问，以及任何子类访问，即使这些子类在不同的包中。
  - 为了便于说明，先简要介绍一下关于Java的继承：
    - 继承允许一个类（称为子类或派生类）获得另一个类（称为父类或基类）的属性和方法。
    - 但是子类能够获得父类哪些属性和方法，由父类这些成员的访问控制决定。
    - 子类能够继承 protected 和 public 成员，但不能继承 private 成员。
    - 如果子类与父类在同一包中，它还可以 继承默认访问级别的成员。

◆(4)保护(protected)

■比如， Stackstructure类继承Linestructure:

```
public class Stackstructure extends Linestructure
```

■Linestructure的show方法访问控制是”默认“，那么:

□如果Stackstructure和Linestructure属于同一个包，则继承后Stackstructure类也就拥有了show方法，它的实例就可以使用show().

□如果Stackstructure和Linestructure不属于同一个包，则继承后Stackstructure类不拥有show方法,它的实例也就不可以使用show()。

◆(4)保护(protected)

■Stackstructure和Linestructure不在一个包中的情况下，如果希望Stackstructure拥有show方法，有两个办法：

□方法一：把Linestructure的show方法的访问控制修改为public。这样的缺点：不光Stackstructure类拥有了show方法，所有其他类也都能够调用show方法，公开范围过大。

□方法二：把Linestructure的show方法的访问控制修改为protected。这样Linestructure的子类，无论是否和它属于同一个包，都拥有了show方法。而其他的类则不能访问show。提高了封装性和细节隐藏。

◆经过修改后的Linestructure类如下:

```
package access.structure2;
public class Linestructure {
    public Linestructure() {
        System.out.println("Linestructure constructor");
    }
    protected void show() { System.out.println("show"); }
}
```

```
C>tree /f
文件夹 PATH 列表
卷序列号为 6A8A-A920
C:.
    StackStructure2.java

└access
    └structure
            LineStructure.java
```

◆(4)保护(protected)

■类Stackstructure2 是它的子类，虽然不在同一个包中，仍然可以访问show().

```
import access.structure.*;
public class Stackstructure2 extends LineStructure {
    public Stackstructure2() {
        System.out.println("Stackstructure2 constructor");
    }
    public void display() { show(); } // 保护的方法:Protected
    public static void main(String[] args) {
        Stackstructure2 x = new Stackstructure2();
        x.display();
        x.show();
    }
}
```

◆如果是命令行编译

- 应该把当前目录设置到StackStructure2.java所在的目录。
- 检查classpath环境变量。如果没有设置，默认就会搜索当前目录。
- 否则如果设置了classpath,必须包含".",不然编译时会找不到符号"LineStructure"

```
C>set classpath
环境变量 classpath 没有定义

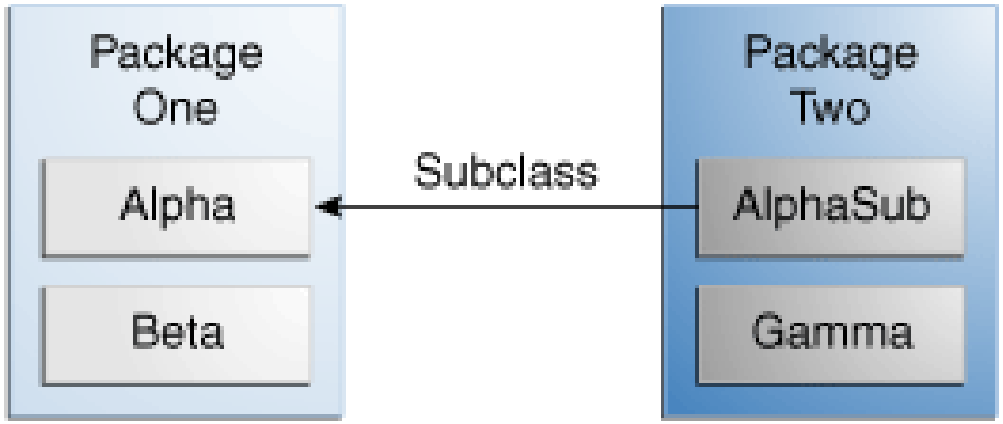
C>javac StackStructure2.java

C>java StackStructure2
Linestructure constructor
Stackstructure2 constructor
show
show
```

4.1.3 Java 的访问控制

类型	private	无修饰	protected	public
同一类	yes	yes	yes	yes
同一包中的子类	no	yes	yes	yes
同一包中的非子类	no	yes	yes	yes
不同包中的子类	no	no	yes	yes
不同包中的非子类	no	no	no	yes

4.1.3 Java 的访问控制



Alpha类的访问控制为public,则其成员的可见性如右面表格:

Visibility

Modifier	Alpha	Beta	Alphasub	Gamma
`public`	Y	Y	Y	Y
`protected`	Y	Y	Y	N
no modifier	Y	Y	N	N
`private`	Y	N	N	N

4.1.4 set 和 get 函数(setters, getters)

◆很多Java程序员认为，所有的成员变量都应该声明为private，这样就可以让程序员完全控制变量的操作。

◆为了让其它类操作该变量，通过public函数来提供对私有变量的读取和修改。这就是Getter 方法和Setter 方法。

◆比如，某个类有一个private的成员变量title, 可以提供setter和getter如下：

```
public String getTitle() {
    return title;
}
public void setTitle( String newTitle ) {
    title = newTitle;
}
```

4.1.4 set 和 get 函数(setters, getters)

◆很多Java程序员认为，所有的成员变量都应该声明为private，这样就可以让程序员完全控制变量的操作。

◆为了让其它类操作该变量，通过public函数来提供对私有变量的读取和修改。这就是Getter 方法和Setter 方法。

◆比如，某个类有一个private的成员变量title, 可以提供setter和getter如下：

```
public String getTitle() {
    return title;
}
public void setTitle( String newTitle ) {
    title = newTitle;
}
```

4.1.4 set 和 get 函数(setters, getters)

◆很多Java程序员认为，所有的成员变量都应该声明为private，这样就可以让程序员完全控制变量的操作。

◆为了让其它类操作该变量，通过public函数来提供对私有变量的读取和修改。这就是Getter 方法和Setter 方法。

◆比如，某个类有一个private的成员变量title, 可以提供setter和getter如下：

```
public String getTitle() {
    return title;
}
public void setTitle( String newTitle ) {
    title = newTitle;
}
```

4.1.4 set 和 get 函数(setters, getters)

◆还可以为setter和getter增加必要的功能，比如：

```
public String getTitle() {
    titleAccessCount++; //记录查询次数.
    return title;
}
public void setTitle( String newTitle ) {
    if ( newTitle == null ) //
        title = "(Untitled)"; // 如果参数为null,
设置默认值
    else
        title = newTitle;
}
```

4.1.4 set 和 get 函数(setters, getters)

- setter和getter方法的命名通常遵循一定的约定，这样的约定增加了代码的可读性，也方便了各种工具和框架的使用。
- 对于一个属性，比如someProperty，setter和getter方法通常会命名为：
  - Getter方法：getSomeProperty
  - 第一个单词是get。之后跟着的是属性名称，其首字母大写。
  - Setter方法：setSomeProperty
  - 第一个单词是set。之后跟着的是属性名称，其首字母大写。
- 这种Java Bean命名约定使得属性名成为了getter和setter方法名的一部分。
- 注意，对于boolean类型的属性，getter方法通常会使用is作为前缀，例如属性boolean active的getter方法通常会命名为isActive()。

4.1.4 set 和 get 函数(setters, getters)

◆推荐遵守这些约定，因为很多Java的框架和库（例如JavaBeans规范、Spring等）都预期你的代码将遵循这样的命名模式。如果你的getter和setter方法不遵循这些约定，可能会导致一些工具或框架无法正确地识别和使用这些方法。

```
private int number;
// Getter for `number`
public int getNumber() {
    return this.number;
}
// Setter for `number`
public void setNumber(int
number) {
    this.number = number;
}
private boolean active;
// Getter for `active`
(boolean type)
public boolean isActive() {
    return this.active;
}
// Setter for `active`
public void setActive(boolean
active) {
    this.active = active;
}
```

4.1.4 set 和 get 函数(setters, getters)

◆关键字this的使用

■如果形式参数名与实例变量名相同，则需要在实例变量名之前加this关键字，否则系统会将实例变量当成形式参数。

■如果形式参数为radius，有一个实例变量也是radius，则需要在成员变量radius之前加上关键字this。代码如下：

```
public void setRadius(int radius){
   this.radius = radius;
}
```

4.1.5 数组和对象

- 数组是是特殊的对象，具有自己独特的语法。
- 诸如 int[ ]或String[ ]之类的数组类型实际上是一个类
- 数组是使用 new 运算符的特殊语法创建的: new String[10]
- 与其他对象变量的情况一样，数组变量不能保存实际的数组，只能是对数组对象的引用。
- 数组对象本身存储于堆中，数组变量的值为 null 时，代表没有实际的数组。

◆如果list的值为null, 则不能访问数组的元素和属性:

```
int[] list=null;
list[0]=1; //NullPointerException
int len=list.length; //NullPointerException
```

◆数组的赋值是引用地址的赋值。

```
int[] newlist=list;
```

赋值之后newlist和list引用的同一个数组。

◆下面代码会输出42而不是17，因为list和newlist地址相同：

```
list = new int[3];
list[1] = 17;
newlist = list; // newlist 指向 list 所引用的数组!
newlist[1] = 42;
System.out.println( list[1] );
```

◆当数组作为参数传递给函数时，传递也是引用。

■参考第三章例子：E0305ArrayParameter.java

4.1.5 数组和对象

- 当数组元素类型是类时，数组中每个元素都是对象的引用。数组创建后，元素的值是引用默认值null。
- 这样的数组元素必须通过new来创建。（对比二位数组每一行的创建）。

```
Student[] classlist;
classlist = new Student[30];
for ( int i = 0; i < 30; i++ ) {
    classlist[i] = new Student();
}
```

◆4.2.1 初始化实例变量

■实例变量可以在其声明时分配一个初始值，就像任何其他变量一样。

```
public class PairOfDice {
    public int die1 = (int)(Math.random()*6) + 1;
    public int die2 = (int)(Math.random()*6) + 1;
    public void roll() {
        die1 = (int)(Math.random()*6) + 1;
        die2 = (int)(Math.random()*6) + 1;
    }
 } // end class PairOfDice
```

■每创建一个对象，它的两个骰子die1和die2都是随机值。就像真的扔了一对骰子一样。

■如果实例变量没有设定初值，那么对象创建后它的值是对应类型的默认值。

4.2.2 构造函数

◆对象是用new运算符创建的。就像这样：

```
PairOfDice dice; // 声明一个类型为 PairOfDice 的变量.
dice = new PairOfDice(); // 创建一个对象，并将对象的引用存储到变量中
```

◆其中的” PairOfDice() “就是构造方法(constructor)。

◆new运算符通过调用构造方法，对对象的实例变量进行了初始化。

4.2.2 构造函数

◆构造方法的特点

- ■没有返回类型，修饰符void也不能有
- ■方法名与类名相同
- ■可以在构造函数定义中使用的修饰符只有访问修饰符 public、private 和 protected。
- ■构造函数不能声明为静态的。
- ■可以有一个或者多个参数，也可以没有参数。
- ■在生成一个对象时，系统会自动调用该类的构造方法为新生成的对象初始化
- ■不能在程序中显式的调用， 只能通过new运算符调用。

4.2.2 构造函数

- 系统提供的默认构造方法
  - 如果在类的声明中没有声明构造方法，则Java编译器会提供一个默认的构造方法
  - 默认的构造方法没有参数，其方法体为空
  - 使用默认的构造方法初始化对象时，如果在类声明中没有给实例变量赋初值，则对象的属性值为对应类型的默认值。
- 可以通过重载定义多个构造方法。
- 如果有自定义构造方法，不管有无参数，则系统不再提供默认构造方法。
- 构造方法参数用来进行对象的初始化。

4.2.2 构造函数


```
public class PairOfDice {
    public int die1; // 显示在第一个骰子的值.
    public int die2; // 显示在第二个骰子的值.
    public PairOfDice() {
        roll(); // 调用 roll()方法来模拟掷骰子
    }
    public PairOfDice(int val1, int val2) {
        // 构造函数通过随机函数设置成员变量的值，大小为 1-6 之间
        die1 = val1; // Assign specified values
        die2 = val2; // to the instance variables.
    }
    public void roll() {
        // Roll 函数通过随机函数设置成员变量的值，大小为 1-6 之间
        die1 = (int)(Math.random()*6) + 1;
        die2 = (int)(Math.random()*6) + 1;
    }
} // end class PairOfDicee
```

4.2.2 构造函数

- 改造后的PairOfDice类，提供了两个构造函数
- PairOfDice(int val1, int val2)在创建对象时可以指定两个骰子的点数。
- PairOfDice()利用roll()函数，产生两个随机点数的骰子。
- 两个构造函数都可以配合new创建对象。
- *说明：无参数构造函数中，调用了实例方法roll()。这里仅仅是为了演示无参数构造函数的使用。在5.1.8可以看到，一般不推荐这种做法，因为系统比较复杂时可能会引起潜在问题。

- 下面例子使用 PairOfDice 类来计算两个对象 firstDice 和 secondDice 在出现相同骰子值之前 roll 函数运行了多少次（也就是投掷了多少次）。该例子也说明可以创建同一类的多个实例

◆首先创建两对骰子，点数都是随机值。

```
public class RollTwoPairs {
    public static void main(String[] args) {
        PairOfDice firstDice; // 第一个引用变量.
        firstDice = new PairOfDice();
        PairOfDice secondDice; // 第二个引用变量.
        secondDice = new PairOfDice();
        int countRolls; //计算骰子的设置次数
        int total1; // 第一对骰子的总和.
        int total2; // 第二对骰子的总和.
        countRolls = 0;
```

◆然后循环投掷直到两队点数一致，输出投掷次数。

```
        do { // 循环投掷骰子，直到两对骰子的总和相等.
            firstDice.roll(); // 模拟掷第一对骰子.
            total1 = firstDice.die1 + firstDice.die2; // 计算
总和.
            System.out.println("First pair comes up " +
total1);
            secondDice.roll(); // 模拟掷第二对骰子.
            total2 = secondDice.die1 + secondDice.die2; // 计
算总和
            System.out.println("Second pair comes up " +
total2);
            countRolls++; // 次数累计.
            System.out.println(); // 输出一个空行.
        } while (total1 != total2);
        System.out.println("It took " + countRolls+ " rolls
until the totals were the same.");
    } // end main()
} // end class RollTwoPairs
```

◆利用new调用构造函数创建对象时，执行步骤如下：

① 首先，计算机在堆中获得一块未使用的内存，大到足以容纳指定类型的对象。

② 初始化对象的实例变量。如果实例变量的声明指定了一个初始值，则计算该值并将其存储在实例变量中。否则，使用默认初始值。

③ 构造函数中的实参（如果有）被求值，并将值赋给构造函数的形参。

④ 执行构造函数主体中的语句（如果有）。

⑤ 返回对象的引用。

◆至此，可以完善之前的Student类如下：

```
public class Student {
    private String name; // 学生姓名.
    public double test1, test2, test3; //三门考试成绩.
    public Student(String theName) {
        // 为生成学生对象定义构造函数;
        // 构造函数需要对象创建者给出初始的学生姓名，并不能为 null.
        if ( theName == null )
            throw new IllegalArgumentException("name can't be null");
        name = theName;
    }
    public String getName() {
        // 为姓名提供一个 Getter 方法.
        return name;
    }
    public double getAverage() {
        // 计算三门课的平均成绩.
        return (test1 + test2 + test3) / 3;
    }
} // end of class Stude
```

4.2.2 构造函数

- 之前的Student版本，调用者在new了Student对象之后，还得通过stu.name=...来设置学生姓名。然后程序员有可能忘记这一条。
- 完善之后，在创建Student对象时候，必须提供学生姓名。而且不允许传递null参数。像这样：
- `std = new Student("张三");`
- 完善之后name的访问控制设置为为private，而且只提供了getter,没有提供setter.这就意味着学生对象一旦创建，在对象生存期间，姓名不能被修改。

4.2.3 final 修饰符

- 改善后的Student类，name在构造方法中一旦设置，就再也不会修改了。这种情况下可以把name声明为final:
- `private final String name; // 学生姓名.`
- final修饰的成员变量，必须在构造对象时初始化。
  - 可以在构造方法中赋值
  - 或者声明的同时设置初始值。
  - 也可以在”初始化块“中初始化。*

4.2.3 final 修饰符

- 下面看看类变量和实例变量同时存在的情况。
  - 实例方法中可以调用类方法，也可以使用类变量。
  - 但是类方法中不能调用实例方法，也不能使用实例变量。
    - 因为实例方法和实例变量，依赖于实例存在。

4.2.3 final 修饰符

- 下面看看类变量和实例变量同时存在的情况。
  - 实例方法中可以调用类方法，也可以使用类变量。
  - 但是类方法中不能调用实例方法，也不能使用实例变量。
    - 因为实例方法和实例变量，依赖于实例存在。
- 比如Student类，希望每个学生都有一个唯一的ID.
  - 增加一个private变量ID， 因为ID一旦设置不再修改，因此修饰为final。
  - 为了每个学生取得一个唯一ID，设置一个类变量nextUniqueID。在Student对象构造时，利用nextUniqueID取得唯一ID.
  - 改进后代码如下：(E0408)

4.2.3 final 修饰符

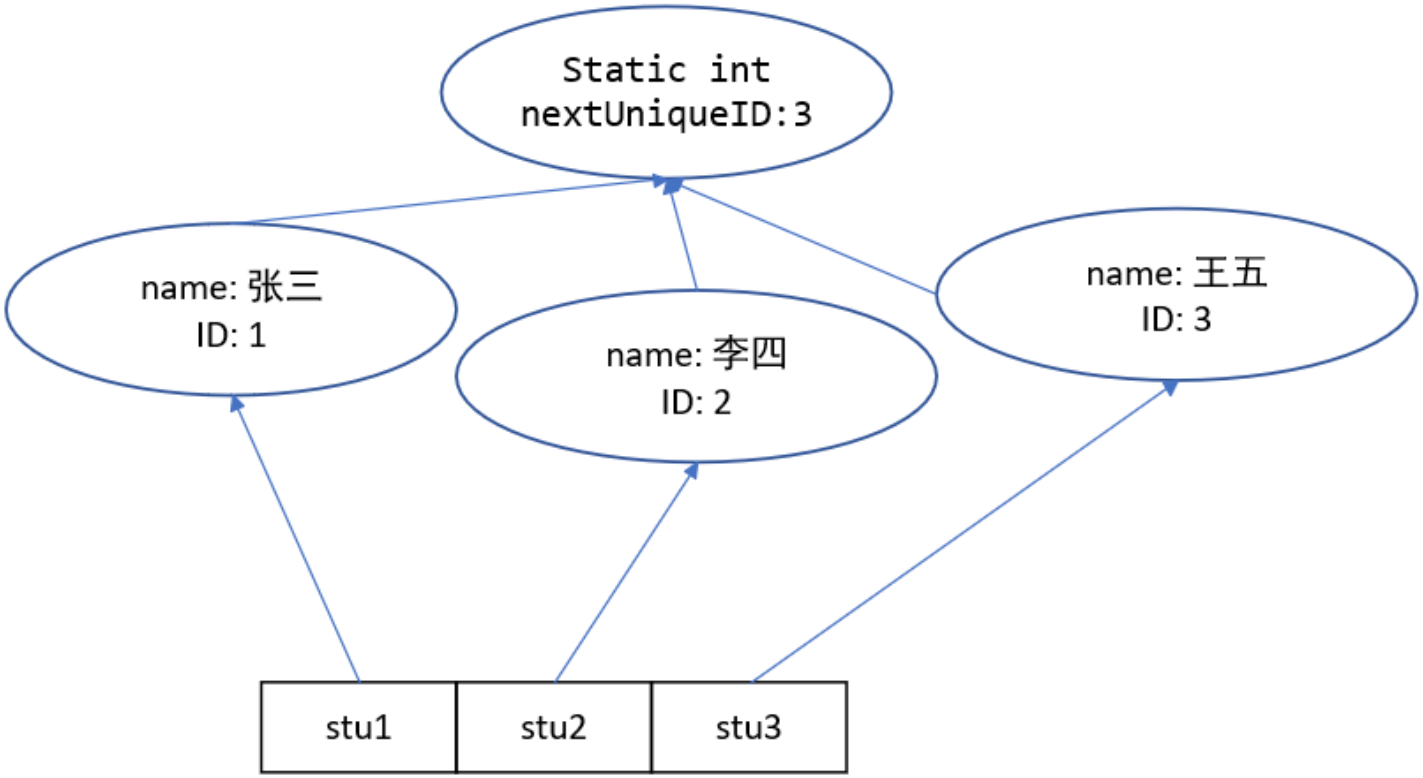
```
public class Student {
    private final
 String name; // 学生姓名.
    public double test1, test2,
test3; // 三门考试成绩.
    private final
int ID; // 学生的唯一 ID 值.
    private static int
nextUniqueID = 0; //跟踪可以得到
的下一个唯一 ID 值
    Student(String theName) {
        // 构造函数，初始化学生
姓名和 ID 值，ID 值保证唯一
        name = theName;
        nextUniqueID++;
        ID = nextUniqueID;
    }
    public String getName() {
        // 为获得学生姓名定义了
Getter 方法
        return name;
    }
    public int getID() {
        // 为获得学生 ID 值提供
Getter 方法
        return ID;
    }
    public double getAverage()
{
        // 计算三门考试课平均值.
        return (test1 + test2
+ test3) / 3;
    }
} // end of class Student
```

4.2.3 final 修饰符

- Student类加载时，nextUniqueID被初始化为0.
- 每次new一个Student时， nextUniqueID加一，然后设置给当前学生。
- 这样每个学生的ID都是唯一的。 nextUniqueID同时也反映了当前曾经new过多少个Student.
  - 这是单线程情况。如果是多线程程序，为了确保ID唯一，还要做更多工作。*

◆比如执行如下三条语句后，内存示意如下图：

```
Student stu1 = new Student("张三");
Student stu2 = new Student("李四");
Student stu3 = new Student("王五");
```



4.2.4 垃圾回收

- 垃圾回收（Garbage Collection，GC）是Java自动内存管理的一部分，它的目的是识别并且删除那些不再被应用程序使用的对象，以便释放和重用内存资源。
- 在C/C++等语言中，开发者必须手动管理内存。如果忘记释放不再需要的对象，将导致内存泄漏。而在Java中，虚拟机自动处理这些问题，减少内存泄漏和内存溢出的风险。
- 回收对象：
  - 当前的代码段不属于对象的作用域
  - 把对象的引用赋值为空

4.2.4 垃圾回收

◆一个对象存在于堆中，它只能通过持有该对象引用的变量来访问。如果一个对象没有引用它的变量，系统就可以自动销毁。

◆比如如下代码：std设置为 null 后， 对象的引用消失了。程序中就不存在任何变量对该对象的引用，这样，程序就永远无法再次使用该对象。此时，系统就可以收回对象std 占有的内存，即回收内存用于其他目的。

```
Student std = new Student("John Smith");
std = null;
```

◆再比如如下代码块，当代码块运行结束后，std就可以被回收。

```
{
    Student std = new Student("John Smith");
    System.out.println(std);
}
```

4.2.4 垃圾回收

- 垃圾回收是根据运行时的具体情况动态决定的。JVM尽可能高效地利用资源，只在需要时执行垃圾回收，以达到优化性能和响应时间的目的。比如：
  - 堆内存不足：当应用尝试分配对象而堆内存不足时，GC将被触发尝试回收内存。
  - 系统调用：可以通过`System.gc()`请求进行垃圾收集，但实际执行取决于JVM。
  - JVM即将退出：当JVM即将退出时，最后一次GC会被执行，以清理所有剩余的对象，然后才真正停止JVM进程。
- JVM具有一套复杂的算法和启发式方法来决定何时触发GC，以及采用哪种类型的垃圾收集策略。

4.2.4 垃圾回收

◆和C\C++的比较：

- 控制水平：C/C++提供了更精细的内存控制能力，而Java则提供了更高级别的抽象。
- 责任分配：Java把内存管理责任交给了JVM的GC，而C/C++则将责任留给了程序员。
- 安全与便利：Java更安全且内存管理相对便利，C/C++中手动内存管理虽更灵活，但错误率较高。
- 实时性：C/C++常用于实时系统，因其可以提供可预测的性能表现；Java的GC可能造成无法预测的暂停。

◆内存泄漏(memory leak)：Java由于有垃圾回收机制，内存泄漏的风险相对较低。尽管如此，内存泄漏在Java中仍然可能发生，例如，对象长时间保持不需要的引用；C/C++：更容易发生内存泄漏，特别是在一个大型复杂程序中，没有正确释放内存会导致泄漏。

4.2.4 垃圾回收

◆现代C++标准（如C++11及以后）利用智能指针（如std::unique_ptr和std::shared_ptr）和资源获取即初始化（RAII）的概念，以便自动管理内存，减少手动内存管理的需求，并降低内存泄漏的风险。尽管如此，这些机制仍然基于程序员的显式控制，而不是像Java中的全自动垃圾回收。*

◆C++ Destructor vs Java finalize*

- C++的类可以定义析构方法（destructor）。 在对象销毁时，析构方法会自动被调用。
- Java类可以提供自己的finalize()方法。
- 旧版本的Java,垃圾回收器回收对象之前，会调用finalize()方法进行清理。
- JDK 9起，finalize()方法已被标注为废弃@Deprecated。因为它的各种缺点,比如执行时机不确定，取决于垃圾收集器决定执行垃圾回收的时间。因此，你无法知道具体何时会调用它。

4.2.5 变量初始化*

- 变量声明时可以设定初值，比如:
  - class Car { int door=4; }
- 可以在构造函数中设置初值, 比如:
  - class Car {Car() { door=5;}
- 实例变量的初始化块。可以把构造函数中共通的处理放到初始化块中。编译器会自动地把初始化块的代码插入到构造函数最前面。
- 初始化块可以写多个。
  - class Car {int door;

```
{   door =3; } //初始化块
    Car() { door=5;}    //
}
```

4.2.5 变量初始化*

- 对于类变量，可以使用静态初始化块。静态初始化块会在类加载时执行。
- 静态初始化块可以有多个，会按照代码顺序执行。
- 例:
  - class Car {
  - static int door=2;
  - static { door =3; } //静态初始化块
  - }
- 执行顺序： 变量初始化和静态初始化块按照代码顺序最先执行。
  - 在加载类时执行一次静态初始化块（之后不再调用）。
  - 在每次初始化实例对象时：先执行非静态初始化块，再执行构造方法。

◆4.3.1 内置类简介

◆字符串操作

- ■如下代码会在堆中创建一个新的字符串对象"Hello!", 而不会在原来的”Hello“对象后面直接拼接字符。
  - □如果s很长，复制s到新对象会花费一些时间。
  - □如果循环中频繁执行这种操作，会产生大量的垃圾需要回收。

```
String s="Hello";
char ch ='!';
String s2 = s+ch;
```

- ■如下代码会在堆中创建一个新的字符串对象"Hello!", 而不会在原来的”Hello“对象后面直接拼接字符

◆StringBuilder

- 利用java.lang.StringBuilder可以进行字符串拼接，而不会发生频繁拷贝和创建新对象。
- 它内部有一个缓冲区，用来存储字符串。只要缓冲区没满，就不会产生新的垃圾。缓冲区满的时候会自动增长。
- 常用的StringBuilder方法
  - append()：将指定的数据添加到当前StringBuilder对象的末尾。
  - insert()：将指定的数据插入到StringBuilder对象的指定位置。
  - delete()和deleteCharAt()：从StringBuilder对象中移除字符。
  - reverse()：将当前字符序列逆序。
  - toString()：将StringBuilder转换为String。

◆StringBuilder可以这样拼接：

```
String s="Hello";
char ch ='!';
String s2 = s+ch;

StringBuilder builder = new StringBuilder("Hello");
char ch='!';
builder.append(ch);
String s2=builder.toString();
```

4.3.1 内置类简介

- StringBuilder不是线程安全的。在单线程环境下效率更高。
- 与StringBuilder类似，Java还提供了一个StringBuffer类。
- StringBuffer是线程安全的，所有的方法都是同步的，适用于多线程并发环境。
- 如果是需要大量的动态的操作字符串，应该选用StringBuffer或者StringBuilder; 否则应该选用String。

4.3.1 内置类简介

- StringBuilder不是线程安全的。在单线程环境下效率更高。
- 与StringBuilder类似，Java还提供了一个StringBuffer类。
- StringBuffer是线程安全的，所有的方法都是同步的，适用于多线程并发环境。
- 如果是需要大量的动态的操作字符串，应该选用StringBuffer或者StringBuilder; 否则应该选用String。

◆日期和时间

- ■Java.util.Date类可以用来处理日期和时间。比如显示当前日期和时间：

```
import java.util.Date;
。。。。。。
System.out.println(new Date());
```

- ■从java 8开始，日期时间处理推荐使用新的java.time包中的类，比如LocalDate, LocalDateTime等。

◆伪随机数

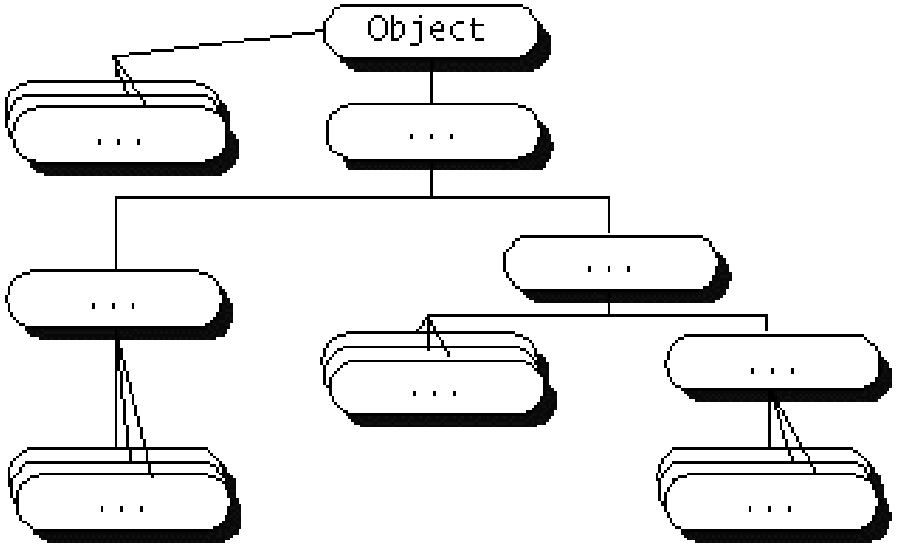
- java.util.Random可以生成伪随机数。
- 标准函数 Math.random()在内部使用 Random 类的对象生成随机数。

```
java.util.Random randGen = new java.util.Random();
int k= randGen.nextInt(N); //生成一个 0 到 N-1 范围内的随机整数
double d= randGen.nextDouble(); //生成一个0.0到1.0之间的随机数
double d2 = randGen.nextGaussian();//生成一个0.0到1.0之间的随机数，随机序列符合高斯分布。
```

4.3.2 Object类

◆Object类

- Java程序中所有类的直接或间接父类，类库中所有类的父类，处在类层次最高点。
- 如果创建了一个类并且没有显式说明其父类，那么它会自动成为 java.lang 包中名为 Object 类的子类。
- 类 Object 定义了一些实例方法，这些方法被其他所有类继承。这些方法可以用于任何对象。



- Object类定义了所有对象必须具有的状态和行为，较主要的方法如下
  - public final Class getClass()
    - 获取当前对象所属的类信息，返回Class对象
  - public String toString()
    - 返回当前对象本身的有关信息，按字符串对象返回
  - public boolean equals(Object obj)
    - 比较两个对象是否是同一对象，是则返回true
  - protected Object clone( )
    - 生成当前对象的一个拷贝，并返回这个复制对象
  - Public int hashCode()
    - 返回该对象的哈希代码值

◆equals 方法

- 由于Object是类层次结构中的树根节点，因此所有其他类都继承了equals()方法
- Object类中的 equals() 方法的定义如下，可见，也是判断两个对象是否同一

```
public boolean equals(Object x) {
    return this == x;
}
```

◆equlas方法的重写

- 要判断两个对象各个属性域的值是否相同（等值判断），则不能使用从Object类继承来的equals方法，而需要在类声明中对equals方法进行重写
- String类中已经重写了Object类的equals方法，可以判别两个字符串是否内容相同

- 比如比较两个Student是否等值，可以在类中重写euqals方法.
- 由于是对Object类中的equals方法进行重写，因此方法定义头必须与Object类中的equals方法完全相同

```
public boolean equals(Object x) {
    if (this.getClass() != x.getClass()) //类型不一致
        return false;
    Student b = (Student) x;
    //根据实际业务，也可以同时比较成绩也都要相等。
    return (this.name.equals(b.getName()));
}
```

- 在equals方法中，还可以利用instanceof运算符判断是否可以继续。
- 比如CollegeStudent类继承自Student类，这样就可以比较学生类的实例和大学生类的实例。
- 按照Java约定，当你重写equals方法时，你必须同时重写hashCode方法。这个约定是为了维护equals和hashCode之间的一致性.
  - 如果两个对象根据equals(Object)方法是相等的，那么调用每个对象的hashCode方法应该产生相同的整数结果。
  - 如果两个对象根据equals(Object)方法是不相等的，那么调用每个对象的hashCode方法不要求必须产生不同的整数结果。然而，程序员应该意识到为不相等的对象产生不同的整数结果可以提高哈希表的性能

4.3.2 Object类


```
@Override
public boolean equals(Object obj) {
    if (this == obj) { //compared with itself
        return true;
    }
    if (!(obj instanceof Student)) {
        return false;
    }
    Student other = (Student) obj;
    return this.ID == other.getID();//仅比较ID
}
@Override
public int hashCode() {
    // Hash code should be consistent with the equals method
    // If two objects are equal according to equals(), they must have
the same hash code
    // Here we return the `studentId` as the hash code, assuming it
is unique for each student
    return ID;
}
```

◆toString()

- Object 类中的实例方法 toString()返回一个 String 类型的值，该值应该是对象的字符串表示形式。
- 当需要对象的字符串形式的内容时，该对象会通过调用其 toString()方法自动转换为 String 类型。
  - 比如，System.out.println(),当参数中有对象时，就会自动调用这个对象的toString();
  - System.out.println(obj) 相当于System.out.println(obj.toString())
- Object类提供的toString()方法，返回的是这样的字符串：

  getClass().getName() + '@' + Integer.toHexString(hashCode())

  比如:

```
Student stu=new Student("张三");
System.out.println(stu);
```

  输出:

```
Student@372f7a8d
```

◆toString()

■可以根据需要，为自己的类提供toString(),代替Object提供的toSting();

```
@Override
public String toString()
{
    return "姓名: "+name+",ID:"+ID;
}
```

■Student类定义了上面的toString()之后，前一页代码输出变成：

```
姓名：张三,ID:1
```

- 基于一个扑克牌游戏，设计几个类，展示面向对象的设计过程以及可重用机制。
- 4.4.1 类的设计
  - 纸牌游戏开始时，给玩家发牌。即把一副扑克牌洗牌后，一张一张发到玩家的手上，每个玩家就会获得一手牌。
  - 游戏进行时，玩家可以出牌，也可以再抓牌。游戏的输赢取决于玩家收到的牌的大小（A、2、3、…、K）和花色（黑桃、方块、梅花、红心）。

◆(1) 设计类

在编写程序的初期，需要确定程序中的类。获取类的最简单的方法是在描述中查找名词作为候选类。对于纸牌游戏，名词如下：

游戏（game）、玩家（player）、一手牌（hand）、

(一副)扑克牌（deck）、牌面(一张牌,card）、牌面的大小（value） 和牌面的花色（suit）。

挑选：

value和suit是简单的数字，可以表示为Card类中的实例变量。

在一个完整的游戏程序中，其余5个都可以表示为类。

我们先关注明显可重用的三个：hand, deck, card.

◆(2) 设计类中的方法。

■在游戏描述中找动词，作为方法候选。

■一副牌Deck需要洗牌(shuffle)和发牌(deal)。因此Deck类就有了两个候选方法：shuffle()和 dealCard()。

■可以从一手牌中出牌，也可以抓牌到手中。这样Hand类就有了两个候选方法：addCard() 和 removeCard()

■对于一张牌, 至少要能确定牌的花色和大小。后面我们会发现更多的实例方法。

4.4.1 类的设计

- (3) 详细设计扑克牌类(Deck)
- 最初创建一副扑克牌对象时，它以某种标准顺序包含 52张牌面。
  - Deck 类将需要一个构造函数来创建一副新的扑克牌。构造函数不需要参数，因为任何新一副牌都与其他一副牌相同。
- 将有一个名为 shuffle()的实例方法，它将 52 张牌重新排列成随机顺序。
- dealCard()实例方法将从一副扑克牌中取得下一张牌，该方法返回类型为Card，因为调用者需要知道正在处理的是什么牌。
- 从一副扑克牌中发下一张牌时，不需要向它提供任何信息，只是简单地取得下一张牌就行，所以 dealCard()没有参数。
- 如果调用dealCard()方法时，一副牌已经发完了会怎样？这种情况可以认为是一种错误，可以抛出一个异常。
- 为了避免上面这条错误，程序需要知道是否还有牌可发。这个应该问Deck对象。Deck对象应该自己知道自己还有多少牌，为此给Deck类设计一个实例方法 cardsLeft()来实现，它返回一副扑克牌中剩余的牌面数量。

◆根据以上设计，Deck类的定义如下：

◆这是一个设计例子，对于使用Deck类来说，知道这些就足够了。

Deck类中的构造函数以及实例方法

```
/* 构造函数，创建一副未洗牌的扑克牌*/
public Deck()

/* 回收所有用过的牌，然后按照随机方式洗牌后，形成一副新的扑克牌*/
public void shuffle()

/*判断一副牌中还剩余多少张牌面*/
public int cardsLeft()

/*发牌.
* @throws IllegalStateException 无剩余牌面时抛出异常.
*/
public Card dealCard()
```

- (3) 详细设计一手牌类(Hand)
- 首次创建玩家的一手牌对象时，对象中牌面数为 0。
- addCard()实例方法将向玩家的一手牌中添加一张牌面。此方法需要一个 Card 类型的参数来指定要添加的牌面。
- 对于 removeCard()方法，需要一个参数来指定要删除的牌面。
  - 删除时，可以指定牌面内容删除（例如，“删除黑桃 A”），也可以指定牌的位置进行删除（例如，“删除手中的第三张牌”）。
  - 因此，removeCard()实例方法就需要实现重载，一个带有 Card类型的参数，指定要移除的牌面，另一个带有 int 类型的参数，指定手中牌面的位置。
- 由于玩家的一手牌中包含可变数量的牌面，因此可以方便地询问玩家的一手牌对象中有多少张牌面。实例方法 getCardCount()返回玩家的一手牌中包含的牌面张数。
- 游戏的时候，有些玩家喜欢把自己手中的一手牌的牌面排列起来，让牌面值相同的牌面紧挨着。因此，可以提供实例方法来对玩家手中的一手牌进行排序。

◆根据卡面分析设计的Hand类如下：

Hand类中的构造函数和实例方法

```
/*构造函数：为一个玩家创建一手牌，初始化为空*/
public Hand()

/*清空玩家手中的全部牌面*/
public void clear()
/*
* 向玩家的一手牌中添加一张牌面
* @throws NullPointerException 牌面为 null 抛出异常
*/
public void addCard(Card c)

/*根据根据指定的牌面，将玩家一手牌中的一张牌面删除*/
public void removeCard(Card c)
/**
* 根据牌面位置将玩家一手牌中的一张牌面删除
* @throws IllegalArgumentException 指定位置不正确时，抛出异常. */
```

◆根据卡面分析设计的Hand类如下（续）:

```
public void removeCard(int position)

/*返回玩家一手牌中剩余的牌面数量*/
public int getCardCount()

/**
* 根据玩家一手牌中的牌面位置返回牌面的大小.
* @throws IllegalArgumentException 指定位置不存在时，抛出异常.
*/
public Card getCard(int position)

/*按照牌面花色排序后，再按照牌面大小排序*/
public void sortBySuit()
/*根据牌面大小排序，牌面大小一样时，再按照牌面花色排序*/
public void sortByValue()
```

4.4.2 Card 类

- Card 类将有一个构造函数，它指定正在创建的牌面的大小和花色。
- 有四种花色，可以用整数 0、1、2 和 3 表示。所以在 Card 类中定义了命名常量来表示四种可能性。例如，Card.SPADES 是代表花色“黑桃”的常数。（这些常量被声明为 public final static int。当然，使用枚举类型可能会更好）。
- 牌面的可能值是数字 1、2、…。 . . , 13，其中 1 代表 A，11 代表 J，12 代表 Q，13 代表 K。同样，定义一些命名常量来表示 A 和面牌的大小。(ACE, JACK, QUEEN, KING)

4.4.2 Card 类

◆根据牌面的大小和花色的情况下构造一张牌面对象。例如，能够使用如下语句调用构造函数：

```
card1 = new Card( Card.ACE, Card.SPADES ); //创建黑桃 A 牌面.
card2 = new Card( 10, Card.DIAMONDS ); // 创建方块 10 牌面.
card3 = new Card( v, s ); // v 和 s 是整数表达式
```

◆Card 对象需要实例变量来表示其大小和花色(value,suit)。

◆这些实例变量设为私有，不能从类的外部更改它们。为此，提供了 getter 方法 getSuit()和 getValue()，以便可以从类外部得到一张牌面的花色和大小。

◆实例变量在构造函数中初始化，之后就永远不会改变，因此，实例变量suit 和 value 声明为 final。

◆一个实例变量被声明为 final，前提是它要么在其声明中被赋予一个初始值，要么在类中的每个构造函数中被初始化。

4.4.2 Card 类

◆添加一些常用的方法，以便能输出人类容易理解的信息。

- 例如，能够将牌面的花色打印为单词“Diamonds”，而不是无意义的代码数字 2。
- 实例方法getSuitAsString()返回花色的字符串表示形式。
- 实例方法getValueAsString() 返回牌面大小的字符串形式。

◆最后，定义实例方法 toString()以返回一个包含牌面大小和花色的字符串，例如“红桃皇后(Queen of Hearts)”。这样就使得下面的两行代码功能是等价的。

```
System.out.println( "你的牌面是" + card );
System.out.println( "你的牌面是" + card.toString() );
```

◆完整的 Card 类参见Card.java.

- 这个类可以重用，重用为程序的设计、编写和测试带来方便。

4.4.3 HighLow 游戏

- 用Card和Deck类来设计一个完整的游戏。
- 游戏命名为 HighLow。一副扑克牌首先被洗牌，然后从中发一张牌面给玩家。
- 玩家预测一副扑克牌中的下一张牌面大小是高于还是低于玩家当前牌面的大小。如果玩家预测正确，则把下张牌发给玩家，并且玩家可以进行另一次预测。
- 当玩家做出不正确的预测时，游戏结束，并根据正确预测的次数计算玩家的分数。

- 代码参考：HighLow.java
  - TextIO.java是用来进行用户输入的，类似于Scanner类。

4.4.3 HighLow 游戏

◆游戏演示:

```
C>java HighLow
Your score in the game is the
number of correct predictions you
make before
you guess wrong.

The first card is the 10 of Spades
Will the next card be higher (H)
or lower (L)?  L
The next card is 2 of Spades
Your prediction was correct.

The card is 2 of Spades
Will the next card be higher (H)
or lower (L)?  h
The next card is 10 of Diamonds
Your prediction was correct.

The card is 10 of Diamonds
Will the next card be higher (H)
or lower (L)?  l
The next card is 6 of Diamonds
Your prediction was correct.

The card is 6 of Diamonds
Will the next card be higher (H)
or lower (L)?  l
The next card is Jack of Clubs
Your prediction was incorrect.

The game is over.
You made 3 correct predictions.

Play again? n

You played 1 games.
Your average score was 3.000.

C>
```

- 一个类表示一组具有相同属性和行为的对象。类通过指定包含在类的每个实例中的属性来确定对象的状态，并通过实例方法来确定行为。
- 面向对象编程真正区别于传统编程的思想是允许类表达对象之间的相似性，这些对象共享它们的一些状态和行为，但不是全部。这种相似性可以用继承和多态来表达。

4.5.1 类继承的概念

◆当需要对现有的类做一些增加或者修改，来满足新的要求时，可以进行类的扩展，也就是建立现有类的子类：

```
public class <subclass-name> extends <existing-class-name>
{
    .
    . // 变更内容或者新增加的内容.
    .
}
```

◆extends 关键字代表类的继承

◆其中 <existing-class-name> 叫基类 (base class)，也称为父类 (parent class, super class)。

◆<subclass-name>称为派生类(derived class)，也称为子类(child class, subclass)。

4.5.1 类继承的概念

- 先了解一下Balckjack游戏怎么回事。Blackjack游戏又称为“21点”。玩家尝试使手牌的总点数尽可能接近21点，但不超过21点。
- 1. 牌的价值：
  - 数字牌：2到10的牌按面值计算； 人物牌（J、Q、K）：每张算作10点。
  - A（Ace）：可以算作1点或11点，视玩家手牌决定。
- 2. 游戏开始：
  - 每位玩家下注后，玩家和庄家都会收到两张牌。在大多数情况下，玩家的两张牌都是面朝上的，庄家则有一张面朝上和一张面朝下。
- 3. 玩家的行动：
  - Hit：要求新增一张牌。
  - Stand：满意当前的手牌，不再增加新牌。
  - Double Down：玩家加倍赌注，只要一张牌，然后必须Stand。
  - Split：如果起始的两张牌相同，可以将他们拆分成两手牌独立玩。
  - Surrender（某些游戏变种中才可用）：玩家放弃手牌，失去半数赌注。
- 4. 庄家的行动：
  - 庄家在所有玩家都完成行动后才揭示第二张牌。
  - 庄家必须要牌直到总点数达到规定值（通常是17），然后必须停牌。

4.5.1 类继承的概念

- 例如，假设编写一个玩纸牌游戏 Blackjack 的程序。可以使用第 4.4 节中开发的 Card、Hand 和 Deck 类。
- 然而，新游戏中一手牌的属性和行为都需要扩展，理由是一手牌需要根据Balckjack规则计算点数
  - 点数 等于每个牌的value相加。
  - 对于牌面是数字的，其 value 就是其上面的数字值。
  - 牌 J、Q 以及 K 的 value 是 10。
  - 牌 A （Ace)的 value 可以为 1或者11。
- 处理这个问题的一种方法是通过添加一个计算一手牌点数的方法来扩展现有的 Hand 类。这是此类的定义：

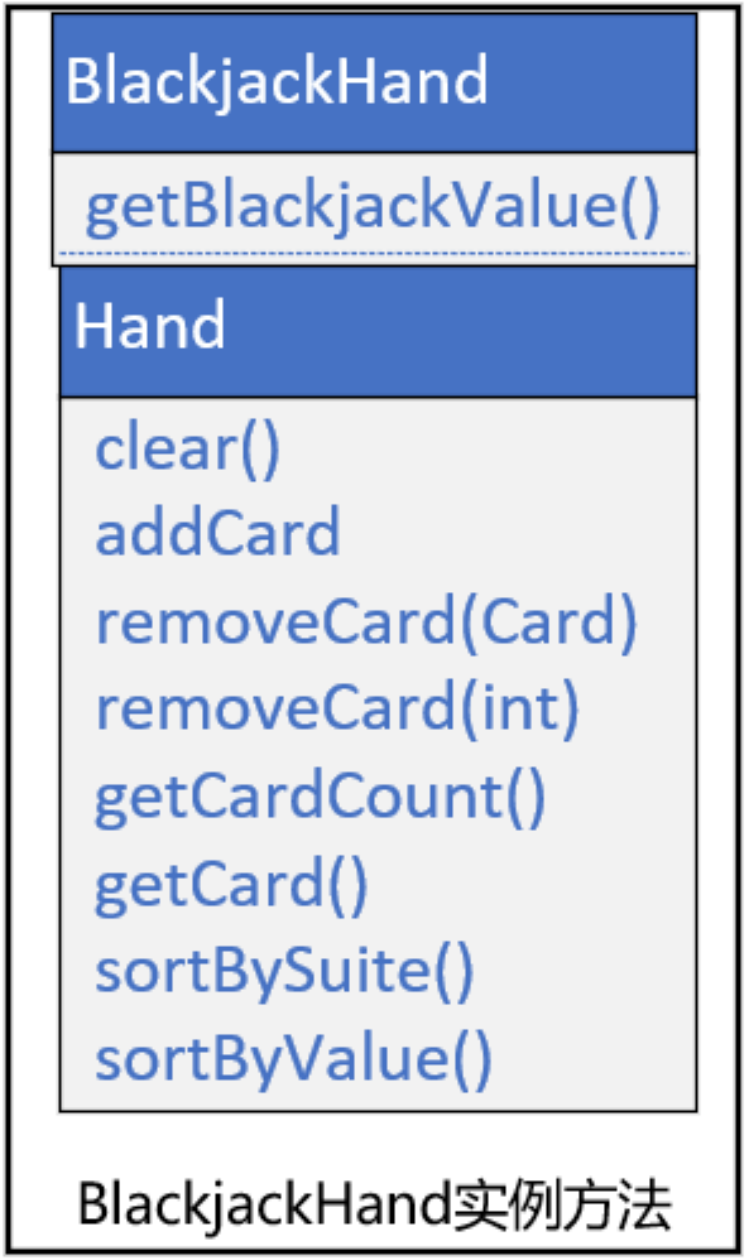
◆根据分析写出BlackjackHand如下：

```
public class BlackjackHand extends Hand {
    public int getBlackjackValue() {
        int val; // 一手牌的计算值.
        boolean ace; // 一手牌中是否包含 A.
        int cards; // 一手牌中牌面数量.
        val = 0;
        ace = false;
        cards = getCardCount(); //该函数定义在 class Hand 中
```

```
        for ( int i = 0; i < cards; i++ ) {//累加每张牌点数
            Card card; // 第 i 张牌面
            int cardVal; //第 i 张牌面的 blackjack 值
            card = getCard(i);
            cardVal = card.getValue(); // 通常值, 1 到 13.
            if (cardVal > 10) {
                cardVal = 10; // 牌面 J, Q,或 K 的值
            }
            if (cardVal == 1) {
                ace = true; // 至少一个 A.
            }
            val = val + cardVal;
        }
        //包含 A 时, 如果值+10 小于 21 的场合, 牌面值+10
        if ( ace == true && val + 10 <= 21 )
            val = val + 10;
        return val;
    } // end getBlackjackValue()
} // end class BlackjackHand
```

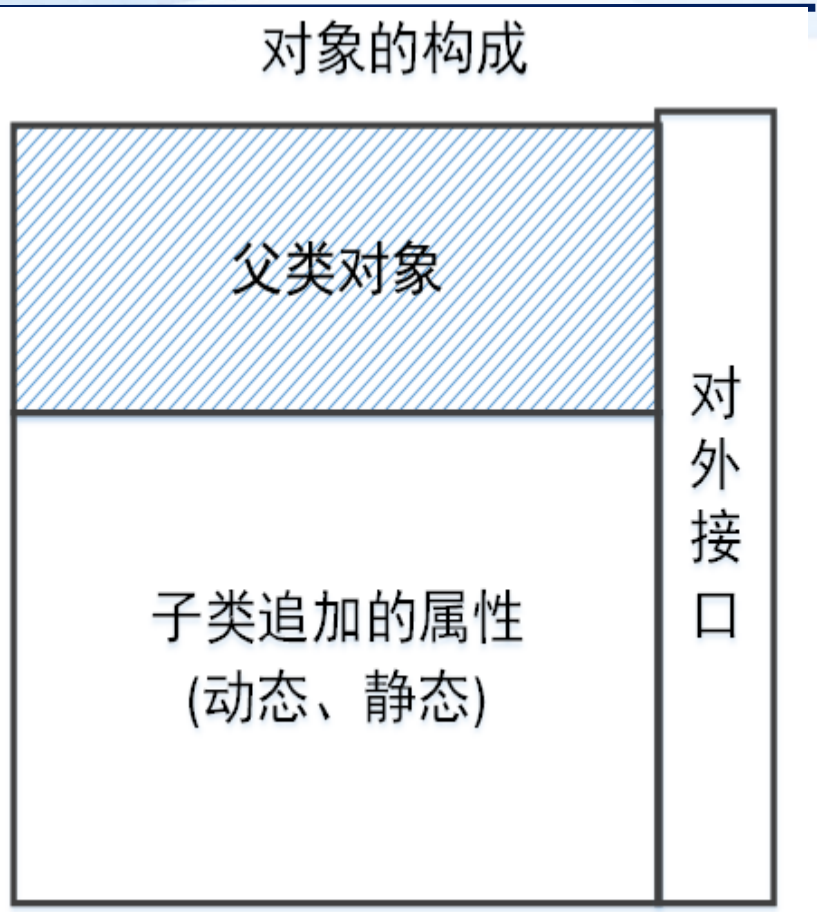
4.5.1 类继承的概念

- BlackjackHand 类型的对象包含 Hand 中定义的所有实例变量和实例方法，以及名为 getBlackjackValue()的新实例方法。
- 例如，如果 bjh 是BlackjackHand 类型的变量，那么以下都是合法的：bjh.getCardCount()、bjh.removeCard(0)和 bjh.getBlackjackValue()。前两个方法在 Hand 中定义，但被 BlackjackHand 继承。



4.5.1 类继承的概念

- 对象的实体构成示意图（假设父类可以实例化，也就是父类不是抽象类)
- Hand 类的变量和方法由 BlackjackHand 继承，除了 Hand 类的`private`变量和方法外，它们可以在 BlackjackHand 的定义中使用。
- 例如，方法 getBlackjackValue() 的定义中调用了父类实例中定义的方法 getCardCount()。



4.5.1 类继承的概念

西北工业大学 NORTHWESTERN POLYTECHNICAL UNIVERSITY

- public 和 private 等访问修饰符用于控制对类成员的访问。
- 当考虑到子类时，还有一个访问修饰符 protected。当 protected 作为访问修饰符应用于类中的方法或成员变量时，该成员可以在定义它的类的子类中使用（直接或间接），但不能在非子类中使用。(有一个例外：protected 修饰的成员也可以被同一个包中的任何类访问)
- 如果一个成员被修饰为protected, 那么就认为它是类实现的部分，而不是对外接口的部分。允许子类修改这部分实现。

4.5.1 类继承的概念

◆考虑利用PairOfDice 创建子类的情况：

◆如果不希望PairOfDice类之外修改骰子值，那么：

■把die1和die2修饰为private, 同时提供public 的getter方法仅允许外部读取骰子值。

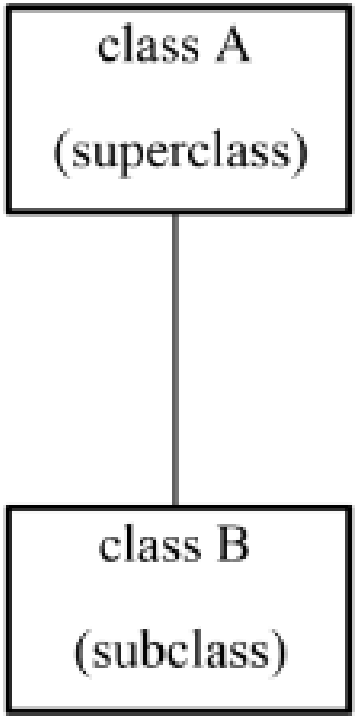
◆如果希望子类可以更改骰子上的数字, 那么:

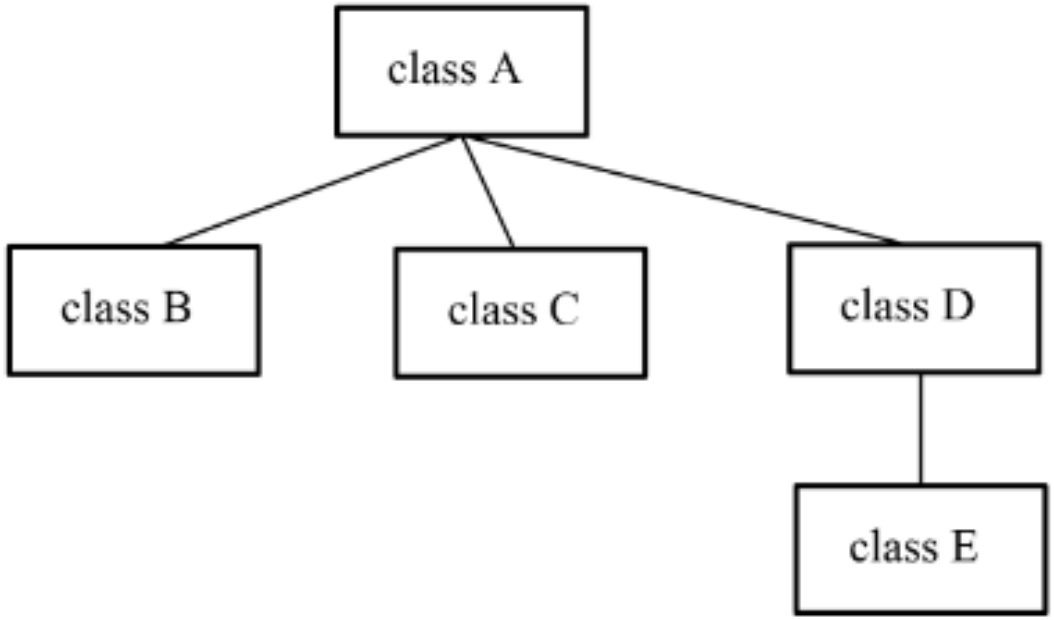
■修改 die1 和 die2 的修饰符为 protected。这样子类能够修改骰子值，而外部又无法直接访问它们。

■或者die1和die2设置为private, 同时提供proteced的setter方法。

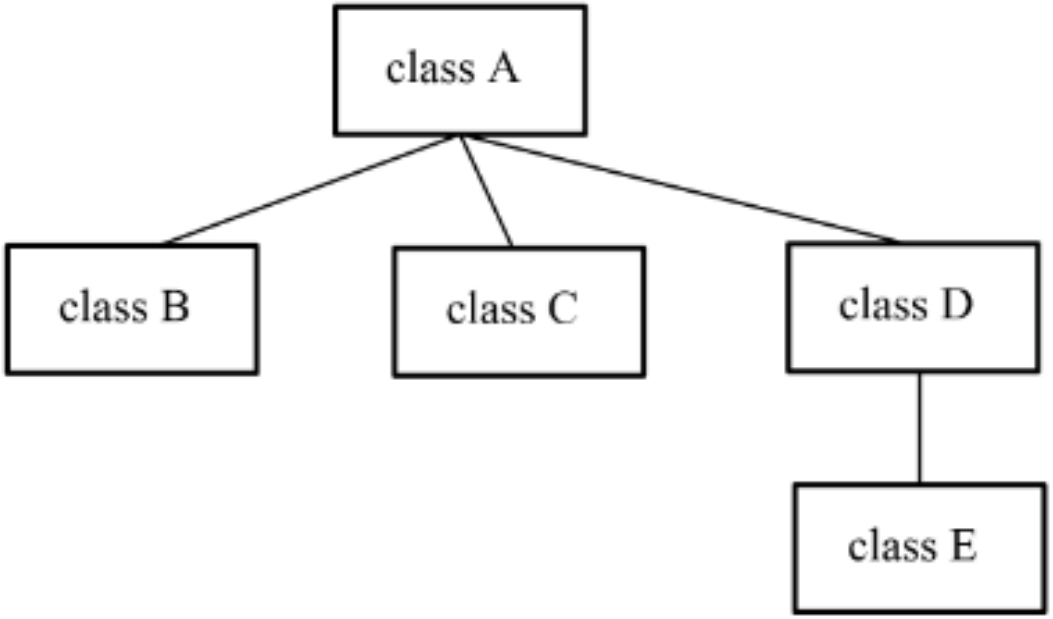
```
public class PairOfDice {
    public int die1; // 第一个骰子的值.
    public int die2; // 第二个骰子的值.
    public PairOfDice() {
      ……
} // end class PairOfDicee
```

- 子类可以添加新的结构和行为，还可以替换或修改继承的行为。
- 一个父类可以有一个子类，一个父类也可以有多个子类
- “兄弟类”，共享从超类继承的属性和行为
- 一个子类只能有一个父类。





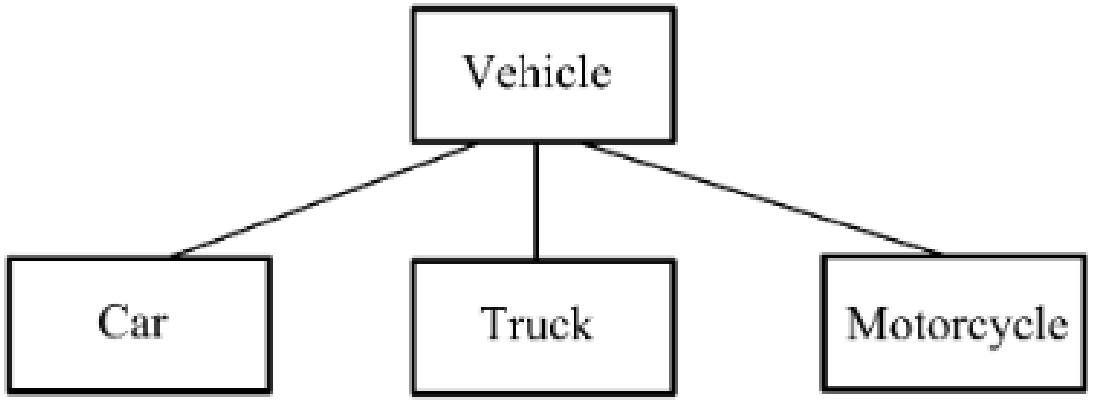
- 继承关系是传递的。比如下图：
- 类 E 继承类 D，类 D 继承类 A，类 E 既有从类 D 那里继承下来的属性与方法，也有从类 A 那里继承下来的属性与方法，还可以有自己新定义的属性和方法。继承来的属性和方法尽管是隐式的，但仍是类 E 的属性和方法。



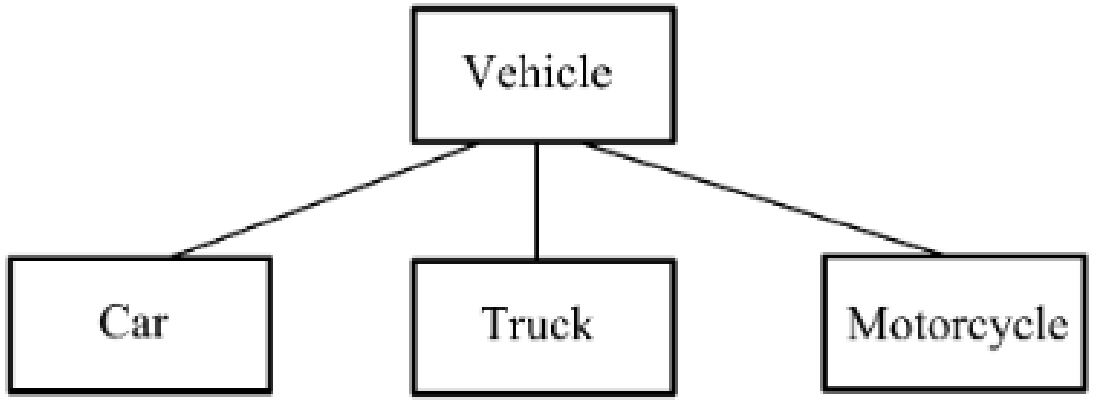
◆例如：车辆管理的程序，车辆包括汽车、卡车和摩托车。该程序可以使用名为 Vehicle 的类来表示所有类型的车辆。由于汽车、卡车和摩托车是车辆的类型，它们将由 Vehicle 类的子类表示。

◆Vehicle 类

- 将包括诸如登记车牌（registrationNumber）和车主（owner）之类的实例变量
- 车辆过户（transferOwnership()）之类的实例方法



- Car 类
  - 可能添加一个实例变量车门数（numberOfDoors）
- Truck 类
  - 可能有车轴数（numberOfAxles）
- Motorcycle 类
  - 可能有一个布尔变量有无车斗（hasSidecar）



4.5.2 继承和类层次结构

◆如果有：Car myCar = new Car();那么这些变量和方法都可以引用：myCar.numberOfDoors，myCar.registrationNumber、myCar.owner，myCar.transferOwnership()

```
class Vehicle {
    int registrationNumber;
    Person owner; //假设类Person 已经定义
    void transferOwnership(Person newOwner) {
    . . .
    }
    int getNumber(){
    return registrationNumber
    }
    . . .
}
```

```
class Car extends Vehicle {
    int numberOfDoors;
    . . .
}
class Truck extends Vehicle {
    int numberOfAxles;
    . . .
}
class Motorcycle extends Vehicle {
    boolean hasSidecar;
    . . .
}
```

4.5.3 成员变量的继承和隐藏

◆(1) 成员变量的继承

■子类可以继承父类的所有非私有成员变量。

■例如，在上面的子类 Car中，其成员变量分别为：

- int registrationNumber; //继承自父类
- Person owner; //继承自父类
- int numberOfDoors

◆（2）成员变量的隐藏

◆在子类，可以重新定义一个与从父类那里继承来的成员变量同名的成员变量

- 子类就具有两个变量名一样的成员变量。
- 当子类执行继承自父类的操作时，处理的是继承自父类的变量，而当子类执行它自己声明的方法时，所操作的就是它自己声明的变量
- 子类中从父类继承来的成员变量不可见，即被子类的同名的成员变量给隐藏了，这一特性称为成员变量的隐藏。

◆比如，在Car类增加一个成员变量：

```
class Car extends Vehicle {
    int numberOfDoors;
    int registrationNumber;
. . .
}
```

◆比如下面代码输出结果：

```
public class testInheritance{
    public static void main(String[] rags){
        Car mycar=new Car();
        mycar.registrationNumber=102154;
        System.out.println("The registrationNumber by
variable is "
        +mycar.registrationNumber);
        firstNum=mycar.getNumber();
        System.out.println("The registrationNumber by
supermethod is "+firstNum);
    }
}
```

The registrationNumber by variable is 102154
The registrationNumber by supermethod is

4.5.4 方法的覆盖和继承

- （1）方法的继承
  - 父类的非私有方法也可以被子类继承。
  - 例如，上面例子中的getNumber、transferOwnership方法，都可以被各个子类继承。

- （2）方法的覆盖
  - 子类也可以重新定义与父类同名的方法，实现对父类方法的覆盖（Override）。
  - 覆盖方法的返回类型，方法名称，参数的个数及类型必须和被覆盖的方法一摸一样
  - 私有方法不能继承，所以不能覆盖。
  - 构造方法不能继承，所以不能覆盖。
  - 静态方法不存在覆盖。
  - 覆盖方法的访问权限可以比被覆盖的宽松，但是不能更为严格（违背“里氏替换原则”）

4.5.5 this 与 super

- （1） this 关键字只能在方法内部使用。this 表示“调用方法的那个对象”的引用。
- this本身是一个引用变量，在需要当前对象引用的表达式，返回值，方法调用中都可以使用。this 关键字常见有三种用法。
  - 1）this.属性名: 区分成员变量和方法参数。

    当函数中的形参与类定义中的成员变量名字完全相同是，this.变量名代表成员变量。

```
public Teacher(String name,double salary,int age)
{
    this.name = name; // 设置名称
    this.salary = salary; // 设置工资
    this.age = age; // 设置年龄
}
```

4.5.5 this 与 super

■2）this.方法名

Java 允许对象的一个成员直接调用另一个成员，可以省略 this 前缀。

```
class Dog {
    public void jump() {
    System.out.println("正在执行 jump 方法");
    }
    public void run() {
    //用 this 调用 jump 方法
    this.jump();
    System.out.println("正在执行 run 方法");
    }
}
public class E0410DogTest {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.run();
    }
}
```

■2）this.方法名

注意： static 修饰的方法中不能使用 this 引用。

static 方法是属于类本身，而不是类的某个具体实例。因此，在static方法中不能使用this关键字，因为this是用来指代当前对象的引用，而在静态上下文中不存在当前对象的概念。

■3）this( )访问构造方法

- this( ) 用来访问本类的构造方法，括号中可以有参数，即调用有参构造方法。
- this( )不能在普通方法中使用，只能写在构造方法中。
- 在构造方法中使用时，必须是第一条语句。
- 可以使得代码简洁，便于维护。

■3）this( )访问构造方法

```
public class Student {
    String name;
    public Student() {// 无参构造
        this("张三");
    }
    public Student(String name) {// 有参构造
        this.name = name;
    }
    public void print() {
        System.out.println("姓名: " + name);
    }
    public static void main(String[] args) {
        Student stu = new Student();
        stu.print();
    }
}
```

4.5.5 this 与 super

◆（2） super 关键字用来在子类中引用父类的成员（变量或方法）或者构造方法。 super只能在方法内部使用。

◆1)super 访问父类中同名成员。

- 当子类隐藏了父类的成员变量，需要使用父类成员变量时。
- 子类覆盖了父类成员方法，需要使用父类成员方法时。
- 不允许使用super.super这样的语法来直接访问祖先类（爷爷类）的成员。Java只允许使用super来访问直接父类（也就是超类或基类）的成员。如果有这样的需求认为是类的设计不合理。

4.5.5 this 与 super

◆比如如下两个类：

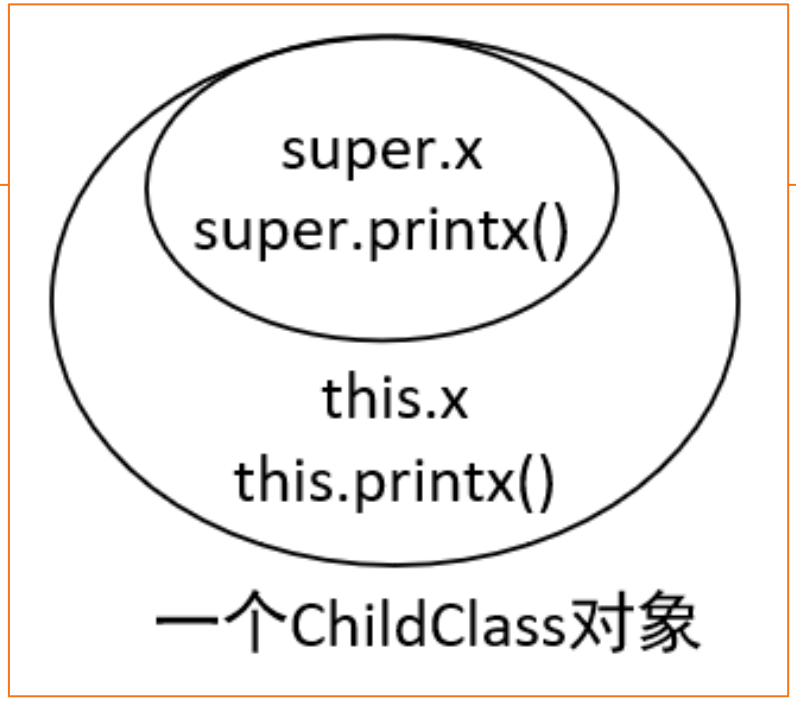
```
class SuperClass {
    int x=10;
    void printx(){
        System.out.println(SuperClass.class.getName()+"
x:"+this.x);
    }
}
class ChildClass extends SuperClass{
    int x=20;
    void printx(){
        super.printx();
        System.out.println("-----------------");
        System.out.println(getClass().getName()+"
this.x :"+this.x);
        System.out.println(getClass().getName()+"
super.x :"+super.x);
    }
}
```

◆验证结果:

```
public class E0411SuperDemo {
    public static void main(String[] args) {
        ChildClass c = new ChildClass();
        c.printx();
    }
}
```

```
C>java E0411SuperDemo.java
SuperClass x:10
----------------
ChildClass this.x :20
ChildClass super.x :10

C>
```



◆2) super调用父类特定构造方法。

■注意点：

□a、在构造函数内部，super（参数列表）必须声明在首行

□b、在构造函数内部，this（参数列表）或 super（参数列表）只能出现一个.

□c、当构造函数中不显式调用 this（参数列表）或 super（参数列表）时，默认会自动调用父类中的无参构造函数。这也就是为什么如果定义了构造方法，最好同时提供无参数构造方法，因为子类可能会需要自动调用父类的无参数构造方法。

4.5.5 this 与 super


```
class Parent {
    private String name;
    // Parent类的构造方法
    public Parent(String name) {
        this.name = name;
        System.out.println("Parent set name: " + name);
    }}
class Child extends Parent {
    private int age;
    // Child类的构造方法，在这里调用Parent类的构造方法
    public Child(String name, int age) {
        super(name); // 使用super调用父类的构造方法
        this.age = age;
        System.out.println("Child seth age: " + age);
    }}
public class E0412SuperConstructor {
    public static void main(String[] args) {
        Child child = new Child("赵老六", 25);
    }}
```

4.6 枚举类型

- 枚举是一个被命名的整型常数的集合，用于声明一组带标识符的常数。枚举在日常生活中很常见，例如一个人的性别只能是“男”或者“女”，一周的星期只能是 7 天中的一个等。当一个变量有几种固定可能的取值时，就可以将它定义为枚举类型。
- 在Java中，枚举是一种特殊的类类型，继承自java.lang.Enum 类。它不仅包含了一组有序的常量（这些常量默认是public static final的），而且这个类本身可以拥有字段、方法和构造函数。
- 每个枚举常量其实都是枚举类型的一个实例，它们都被定义为该类型的一个静态成员。枚举常量之间使用逗号分隔，通常它们都是大写的。与其他普通的Java类相比，
  - 枚举类型有如下特点：
  - 枚举类型的构造函数总是私有的。
  - 枚举常量通常被直接访问，无需实例化枚举类型。
  - 枚举类型可以实现接口。
  - 可以在枚举类型中定义抽象方法，并在枚举常量中提供不同的实现。

4.6.1 枚举定义

◆定义枚举语法如下：

```
<enum-modifiers> enum <enum-name> {
    <enum-body>
    // 枚举体，可以包含字段、构造器、方法等。
}
```

- <enum-modifiers>表示枚举的修饰符主要是访问控制符，比如括 public 、 private ；
- <enumname>表示声明的枚举名称；
- <enum-body>表示枚举的成员，任意两个枚举成员不能具有相同的名称，多个枚举成员之间使用逗号分隔。
- 枚举成员通常是大写的，它们实际上是枚举类型的静态实例。
- 枚举体还可以包含构造方法，字段，方法等。

（也就是一个特殊的类，定义这个类同时，也生成了这个类的几个特殊静态实例：枚举成员）

4.6.1 枚举定义

◆定义枚举语法如下：

```
<enum-modifiers> enum <enum-name> {
    <enum-body>
    // 枚举体，可以包含字段、构造器、方法等。
}
```

- <enum-modifiers> 表示枚举的修饰符主要是访问控制符，比如括 public 、 private ；
- <enumname>表示声明的枚举名称；
- <enum-body>表示枚举的成员，任意两个枚举成员不能具有相同的名称，多个枚举成员之间使用逗号分隔。
- 枚举成员通常是大写的，它们实际上是枚举类型的静态实例。
- 枚举体还可以包含构造方法，字段，方法等。

（也就是一个特殊的类，定义这个类同时，也生成了这个类的几个特殊静态实例：枚举成员）

◆测试简单的枚举类型：

```
enum SexEnum {
    male,female;
    }
enum Color {
    RED,BLUE,GREEN,BLACK;
}
public class E0413EnumSimple {
    public static void main(String[] args) {
        System.out.println(SexEnum.female);
        System.out.println(Color.BLACK);
    }
}
```

```
C>javac E0413EnumSimple.java

C>java E0413EnumSimple
female
BLACK
```

4.6.2 基本枚举功能

- 取得枚举值的数组
  - public static T[] values()
- 取得枚举值对应的字符串
  - public String toString()
- 取得对象在枚举类型中的索引
  - public final int ordinal()
- 字符串转换为枚举值。
  - public static T valueOf(String)

4.6.2 基本枚举功能


```
enum Shrubbery { GROUND, CRAWLING, HANGING }
public class E0414EnumShrubbery {
    public static void main(String[] args) {
        for(Shrubbery s : Shrubbery.values()) {
            System.out.println(s + " ordinal: " + s.ordinal());
            System.out.print(s.compareTo(Shrubbery.CRAWLING) + "
");
            System.out.print(s.equals(Shrubbery.CRAWLING) + " ")
            System.out.println(s == Shrubbery.CRAWLING);
            System.out.println(s.getDeclaringClass());
            System.out.println(s.name());
            System.out.println("----------------------");
        }
        // 从字符串名称生成枚举值:
        for(String s : "HANGING CRAWLING GROUND".split(" ")) {
            Shrubbery shrub = Enum.valueOf(Shrubbery.class, s);
            System.out.println(shrub);
        }
    }}
```

◆执行结果如下：

- compareTo()返回结果是-1,0或者1（根据序号比较)
- name()和toString()返回结果一样。但是name不能重写。

```
C>javac -encoding UTF-8 E0414EnumShrubbery.java

C>java E0414EnumShrubbery
GROUND ordinal: 0
-1 false false
class Shrubbery
GROUND
----------------------
CRAWLING ordinal: 1
0 true true
class Shrubbery
CRAWLING
----------------------
HANGING ordinal: 2
1 false false
class Shrubbery
HANGING
----------------------
HANGING
CRAWLING
GROUND
```

4.6.3 向 enum 类添加方法*

◆枚举类型的构造方法必须是私有的。

```
public enum E0415EnumUniversity {
    //必须先定义实例，然后再定义方法:
    WEST("西北工业大学,位于西安市"),
    NORTH("北京大学,位于北京市"),
    EAST("东北大学,位于沈阳市"),
    SOUTH("南京大学,位于南京市");
    private String description;
    //构造函数必须是包或私有访问:
    private E0415EnumUniversity(String description) {
        this.description = description;
    }
    public String getDescription() { return description; }
    public static void main(String[] args) {
        for(E0415EnumUniversity witch : E0415EnumUniversity.values())
            System.out.println(witch + ": " +
witch.getDescription());
    }
}
```

```
C>java -Dfile.encoding=UTF-8 E0415EnumUniversity.java
WEST: 西北工业大学,位于西安市
NORTH: 北京大学,位于北京市
EAST: 东北大学,位于沈阳市
SOUTH: 南京大学,位于南京市
```

4.6.4 switch 中的 enum 使用*

◆使用枚举还可以使 switch 语句的可读性更强，例如以下示例代码：

```
enum Signal {    GREEN,YELLOW,RED    }
public class TrafficLight {
    Signal color = Signal.RED;
    public void change() {
        switch(color) {
            case RED:
                color = Signal.GREEN; break;
            case YELLOW:
                color = Signal.RED; break;
            case GREEN:
                color = Signal.YELLOW; break;
        }
    }
}
```

4.6.5 枚举类型和单例*

- 枚举类型会由JVM在加载的时候，实例化枚举对象，你在枚举类中定义了多少个枚举值，就会实例化多少个枚举对象。
- JVM为了保证每一个枚举类元素的唯一实例，是不会允许外部进行new的，所以会把构造函数设计成private，防止用户生成实例，破坏唯一性。
- 在Java中，使用枚举类型实现单例是一种线程安全且效率高的方法。枚举保证任何枚举常量都是单例。Java 的枚举单例实现简单，可以防止通过反射或者序列化破坏单例模式的唯一性。

```
public enum Singleton {
    INSTANCE;

    // 添加需要的操作和方法
    public void doSomething() {
        // Do something
    }
}
```